115-1 TAICA《生成式 AI：文字與圖像生成的原理與實務》

04 LLM只是在猜下一個字？

簡單核心與驚人效果

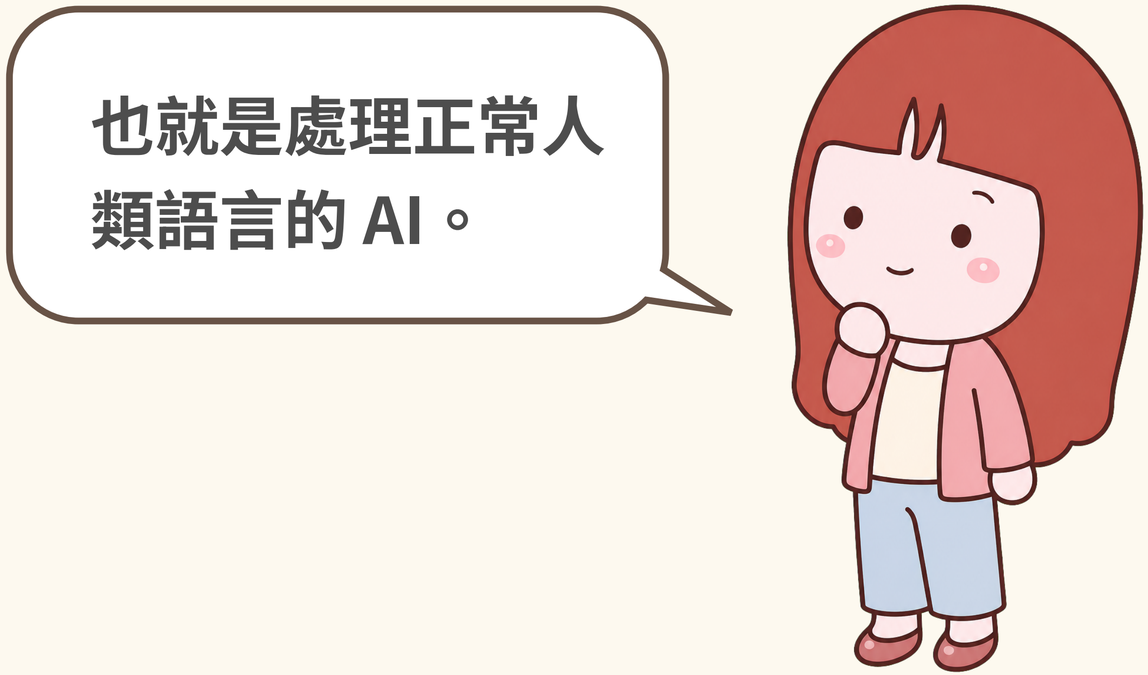
政治大學應用數學系 蔡炎龍

01.

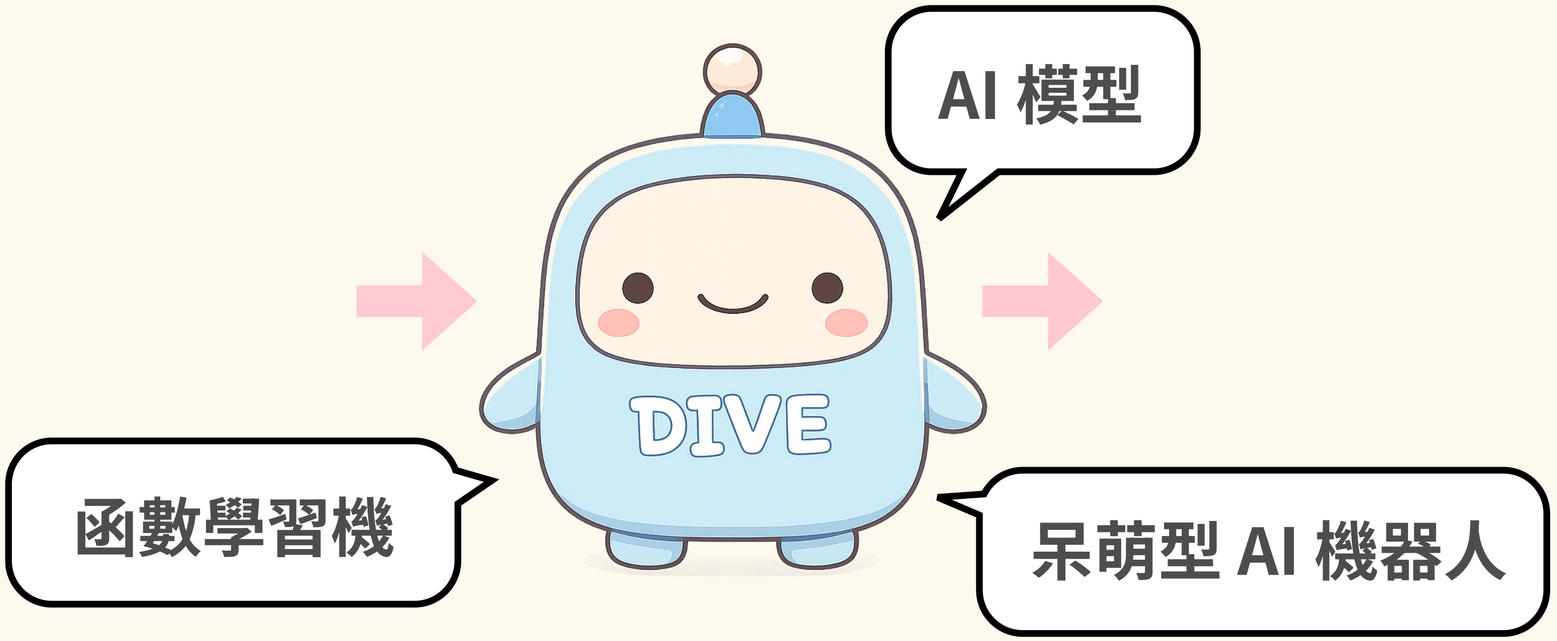
文字生成 AI 的原理

自然語言處理 (NLP)

Natural Language Processing



現代的 AI 模型只是一個呆萌 AI 機器人



我們要告訴它輸入是什麼、輸出是長什麼樣子

AI 最重要的是要「問個好問題」

深度學習會專注在
「我們怎麼問這個問題?」

整個判斷由電腦做, 所以
一般要**大量資料**!

我們可能會這樣想...

客戶說的話

客服人員的回覆

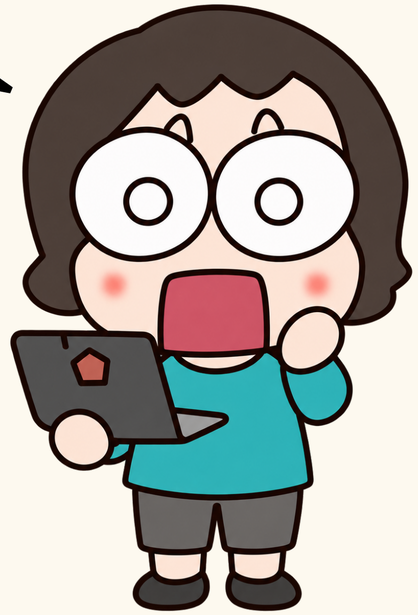
對話機器人不就該用對話來訓練?

但這樣設計會有問題...

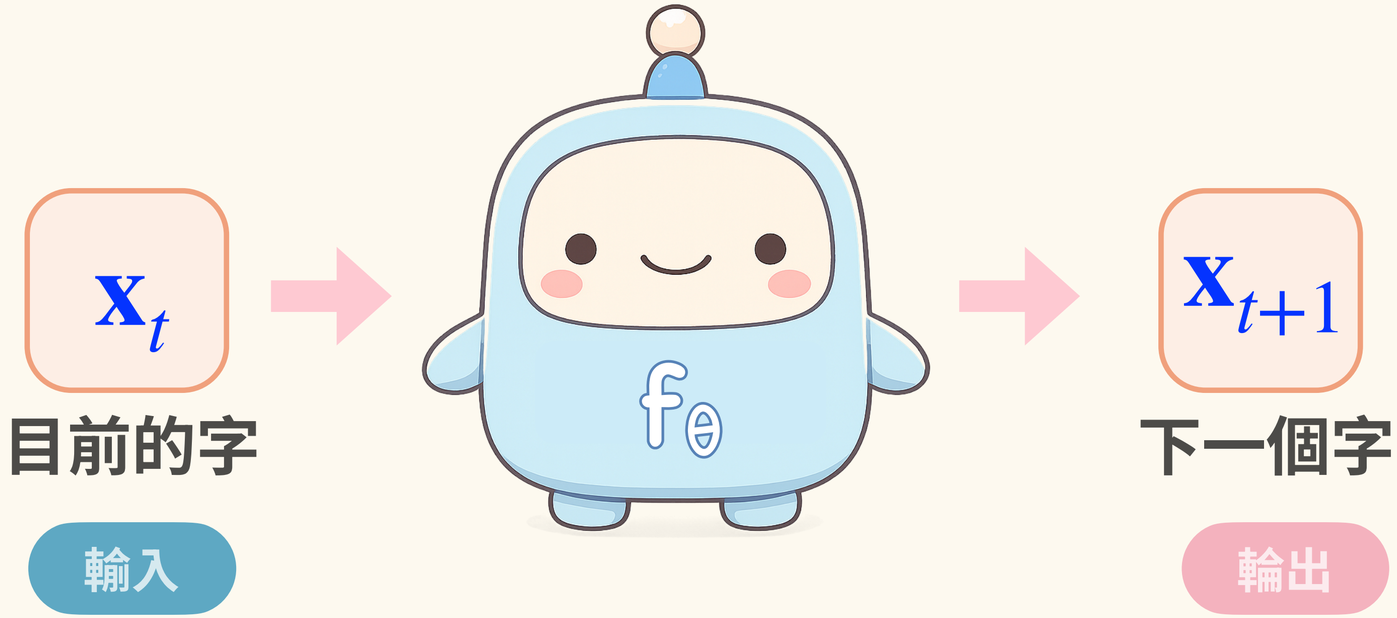
結果不是...

那到底怎麼設計的呢?

看到答案大家的驚呆了!



就是前一個字預測下一個字的模型



看到前面一個字、預測下一個字的 AI 機器人

這樣世界上所有句子都可以拿訓練資料

炎龍老師很帥

x_1 x_2 x_3 x_4 x_5 x_6

(炎, 龍), (龍, 老), (老, 師), (師, 很), (很, 帥)

x_1 y_1 $(= x_2)$ x_2 y_2 $(= x_3)$ x_3 y_3 $(= x_4)$ x_4 y_4 $(= x_5)$ x_5 y_5 $(= x_5)$

但這不是個函數!

比如說, “今天天氣很好。” 這句話拿去訓練, 輸入我們的函數就應該是...

f_θ(“今”) = “天”
f_θ(“天”) = “天”
f_θ(“天”) = “氣”
f_θ(“氣”) = “很”
f_θ(“很”) = “好”

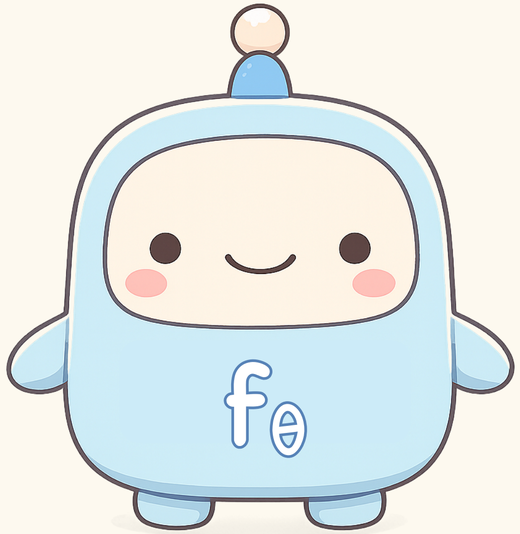
實際上我們用的是「有記憶的」神經網路

$\mathbf{x}_t$

目前的字

前面字 $\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_{t-1}$ 的「記憶」，會融合成一個向量。

$\mathbf{h}_{t-1}$



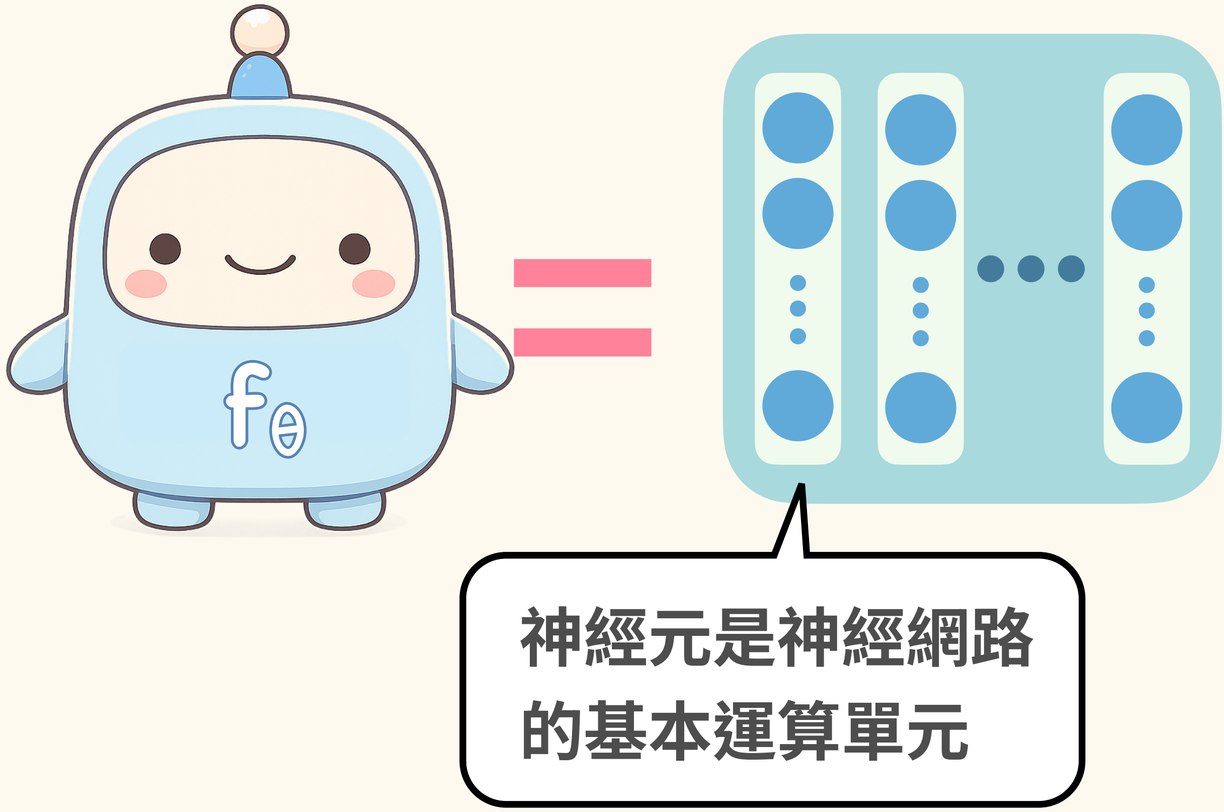
$\mathbf{x}_{t+1}$

下一個字

輸入

輸出

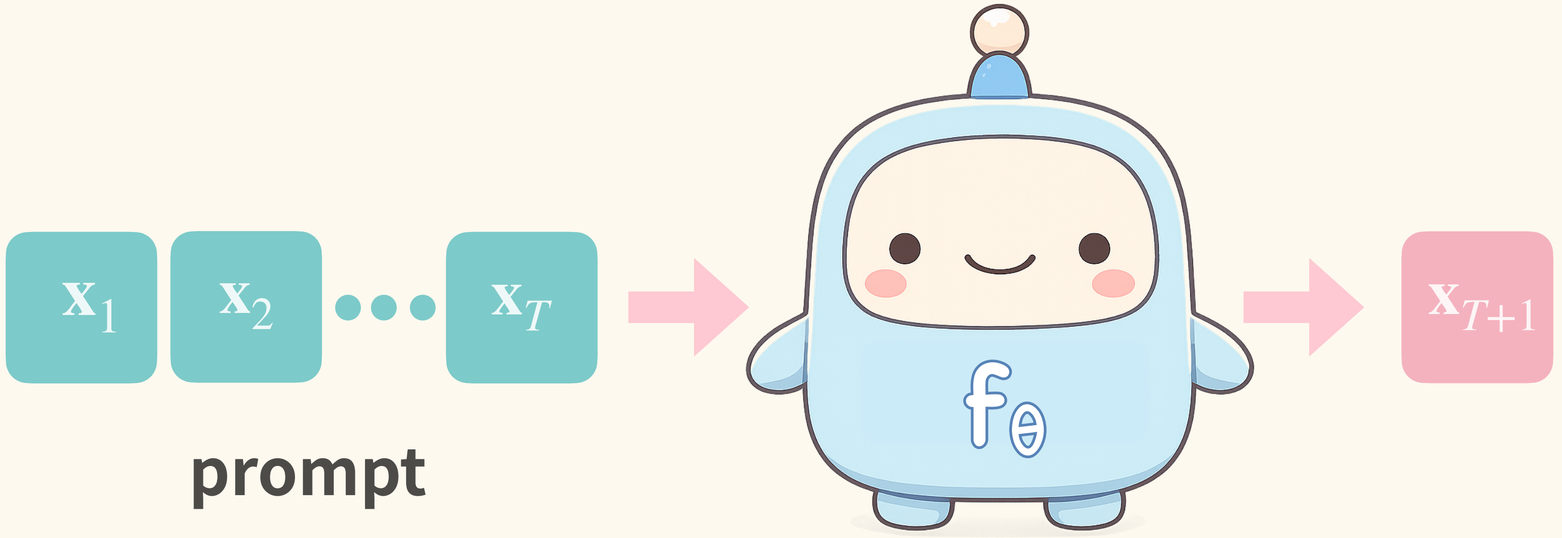
用「神經網路」，輕輕鬆鬆可以打造這個機器人!



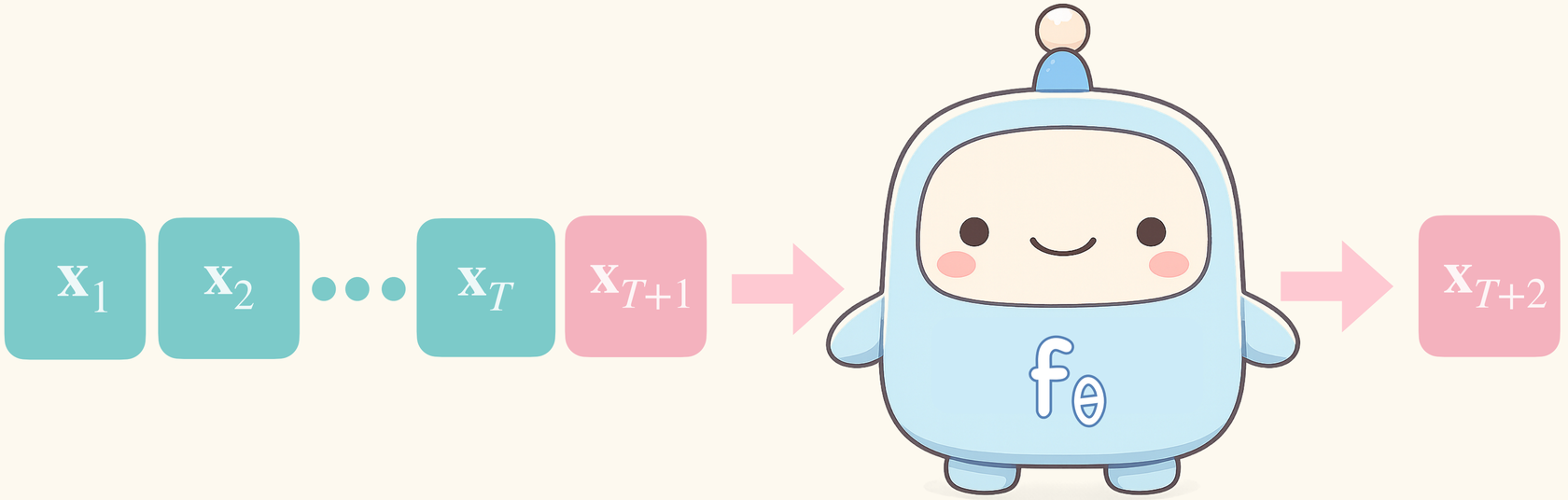
基本上就是建一個個隱藏層, 每層放上若干個「神經元」, 放法差不多就 3+1 種:

- DNN: 標準全連結
- CNN: 圖形辨識很強
- RNN: 有記憶的結構
- Transformers

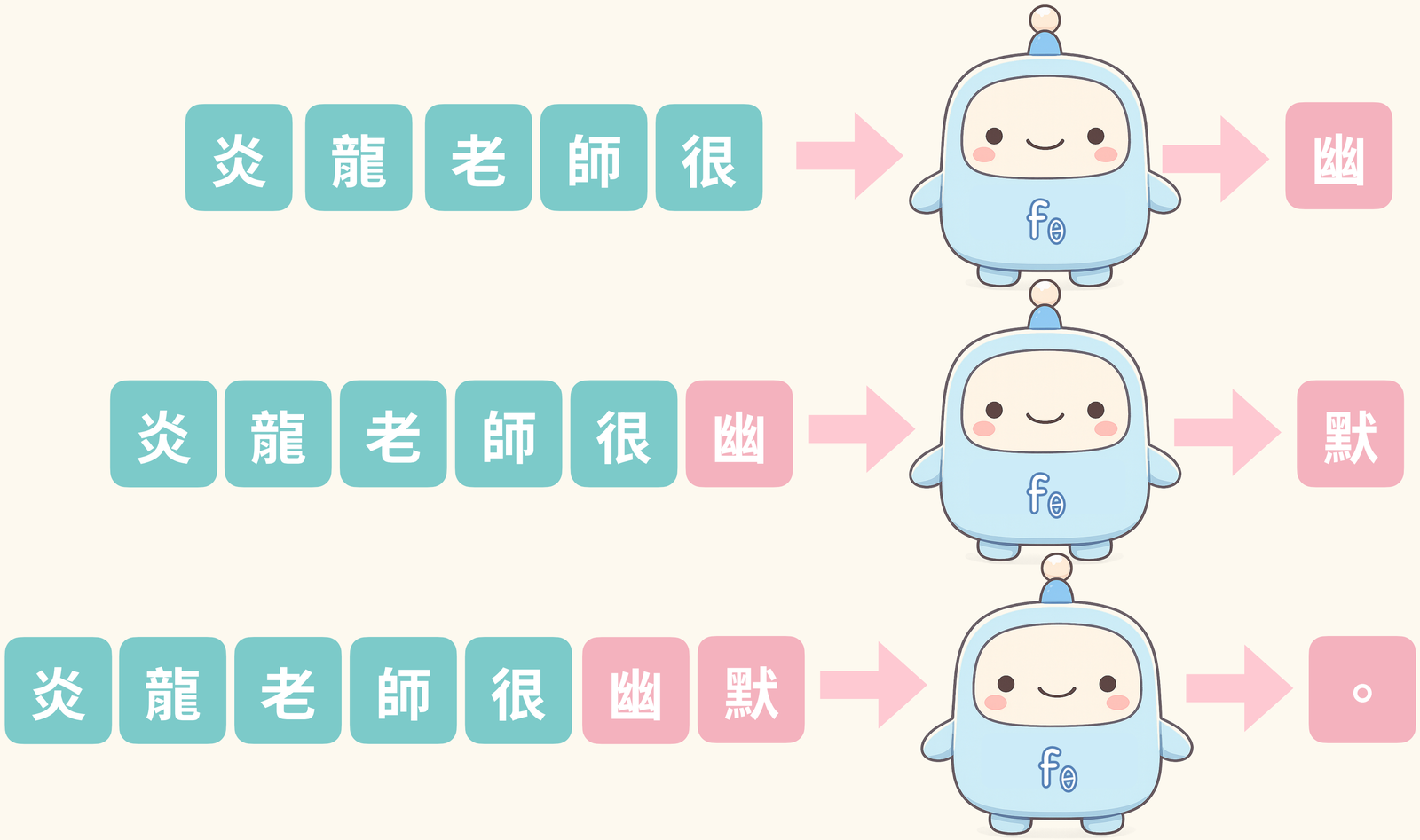
可以想成是一堆字去預測下一個字的模型



然後就可以一路下去...



例如 (Tulu 3 70B)



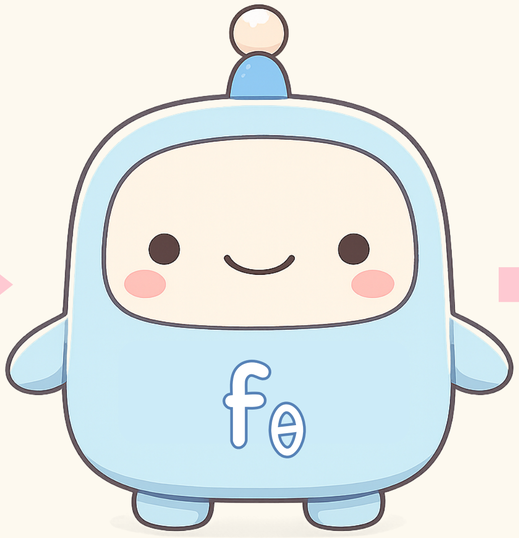
電腦只會處理數字

我們希望語言模型可以生成的字, 都給一個編號。

帥	幽	有	熱	受
87	94	112	174	211

模型其實是把每個可能的字評分

炎 龍 老 師 很



帥	1.40	87
幽	0.03	94
有	1.71	112
熱	−0.73	174
受	0.95	221

模型其實是把每個可能的字評分

		指數轉換	計算佔比
帥	1.40	4.05	30%
幽	0.03	1.03	7%
有	1.71	5.53	40%
熱	−0.73	0.48	4%
受	0.95	2.59	19%

87
94
112
174
221

softmax

語言模型不知它在做什麼，只覺得下一個字放什麼順就放

炎 龍 老 師 很 →

小總結

- 文字生成 AI 只是預測下一個字的呆萌型 AI 機器人。
- 訓練目標就是要把它訓練成「很會接話」的唬爛王。

瞭解原理，就能用好 ChatGPT 等 LLM

提供足夠的資訊。

清楚的指引要做什麼。

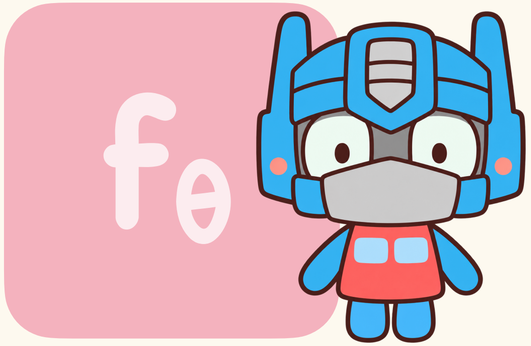
02.

RNN 和變型金剛

兩種有記憶的神經網路

RNN

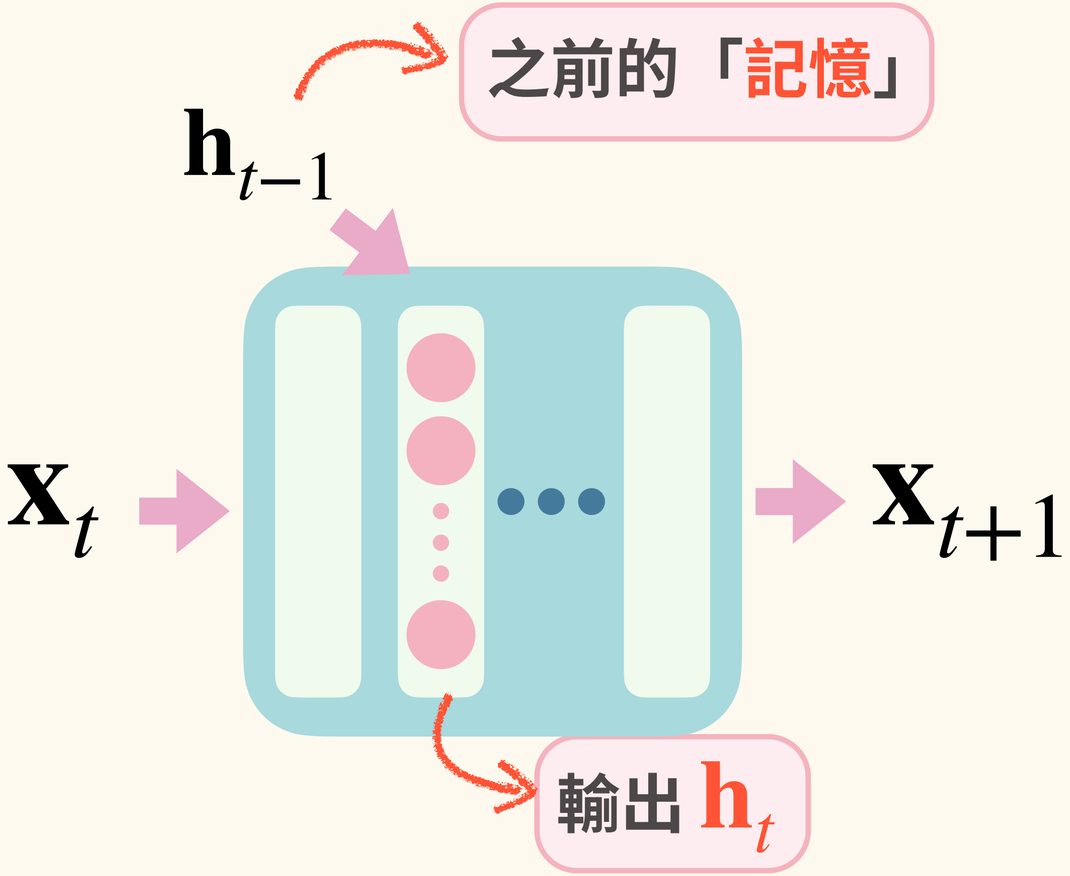
遞歸神經網路



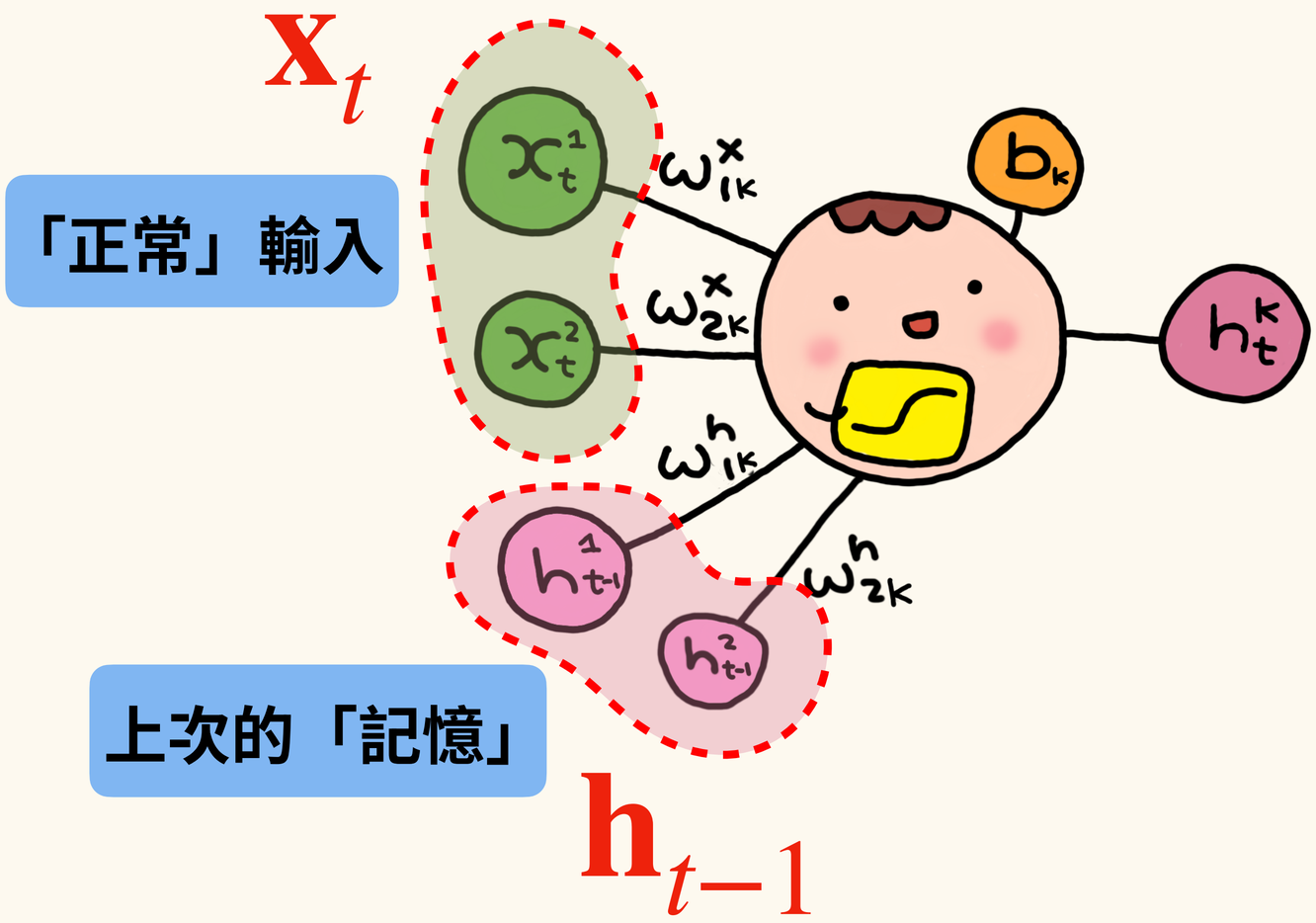
Transformer

變型金剛

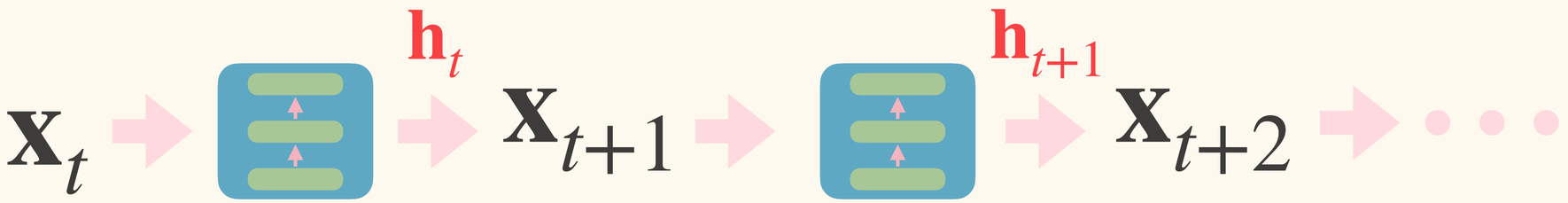
RNN 層會導入上次的記憶



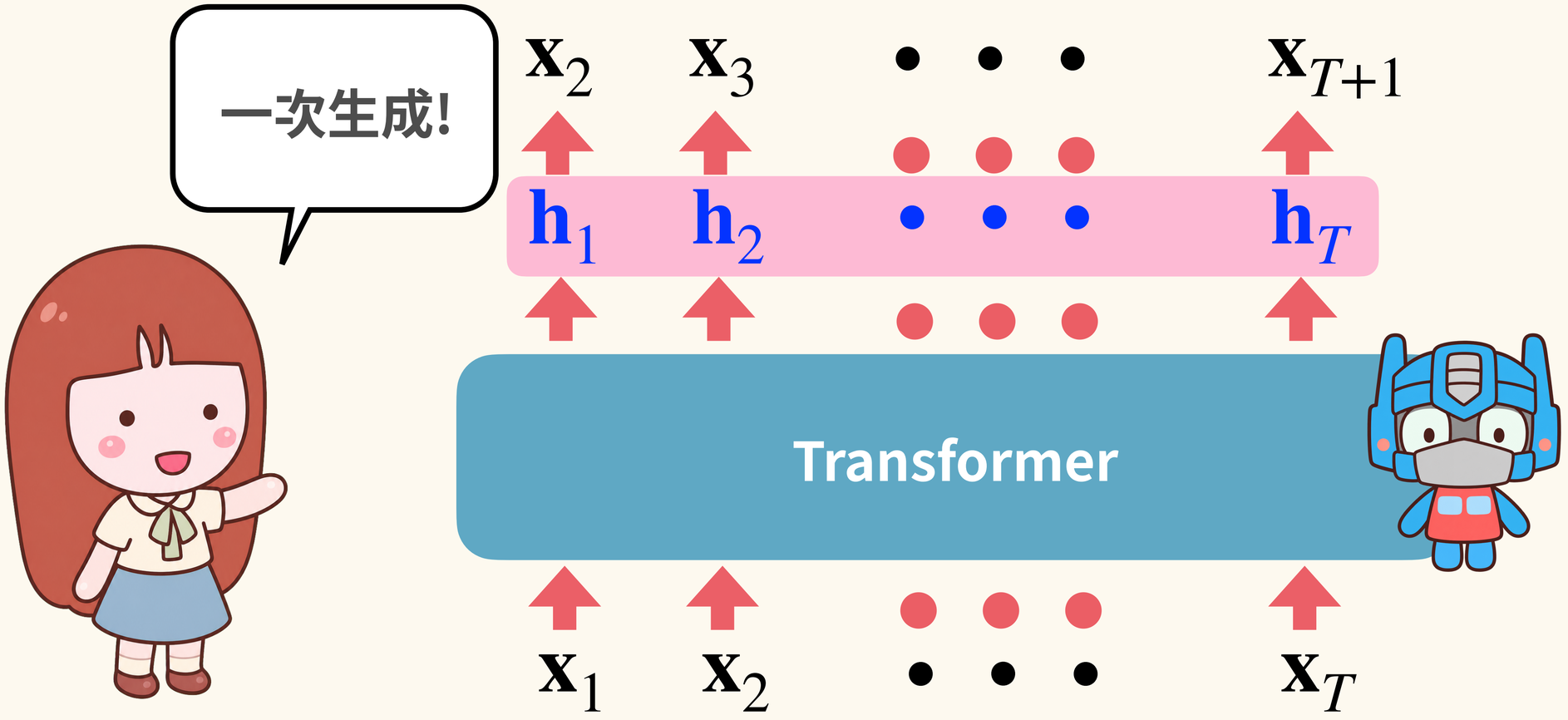
RNN 神經元和正常的其實一樣



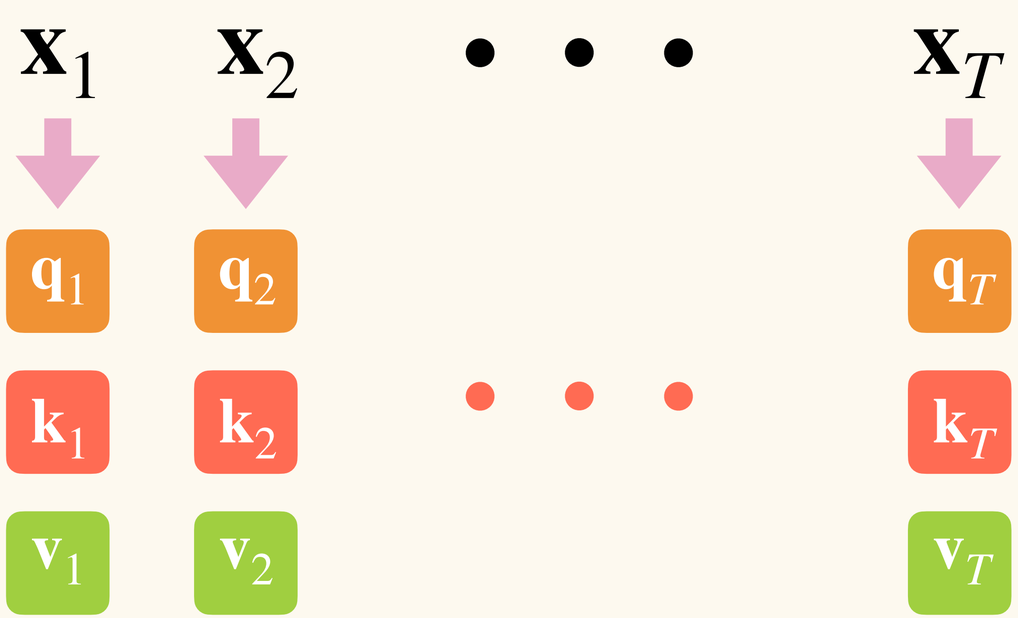
RNN 缺點是遞迴型的運算



Google 想到一次生成各時間點的「記憶」



每個字生出 Q, K, V 三個特徵代表向量



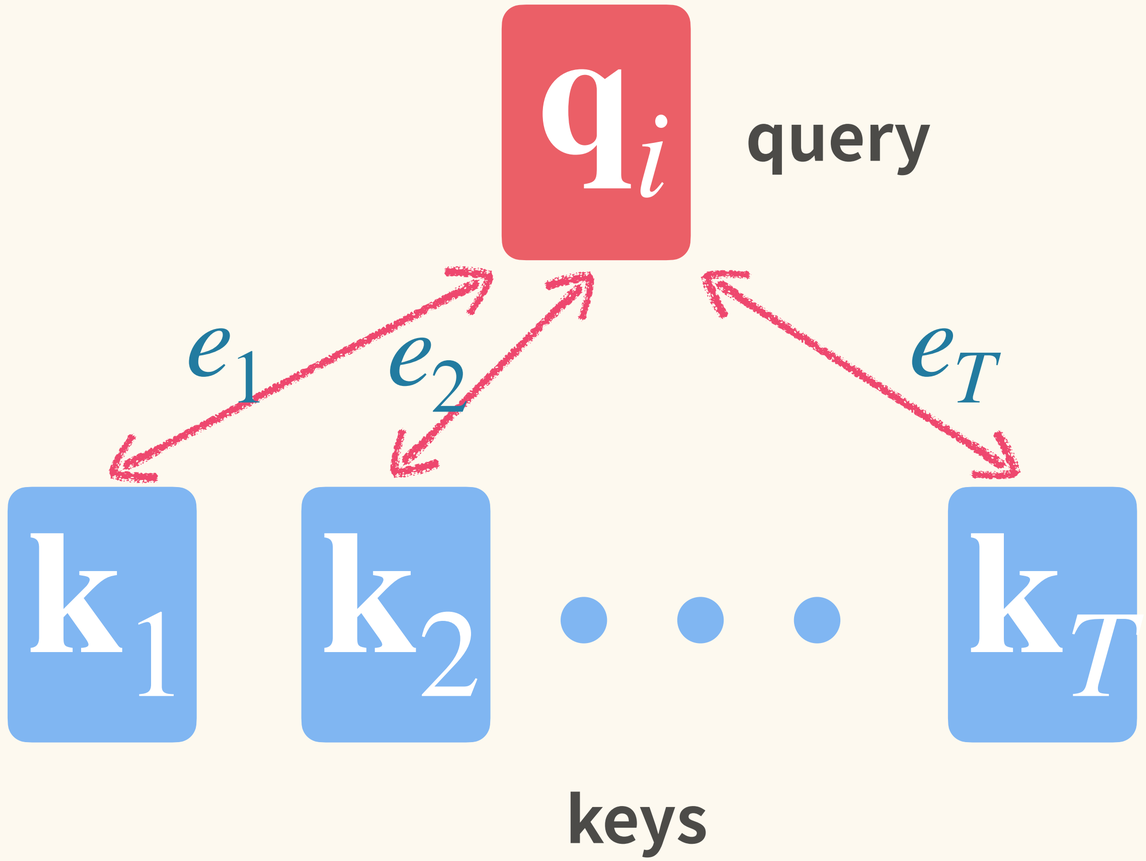
其實只是作線性轉換。

$$\mathbf{q}_i = \mathbf{x}_i W^Q$$

這學習來的

* Google 深愛列向量, 所以線性轉換矩陣是乘在後面。

作所謂的 self-attention!



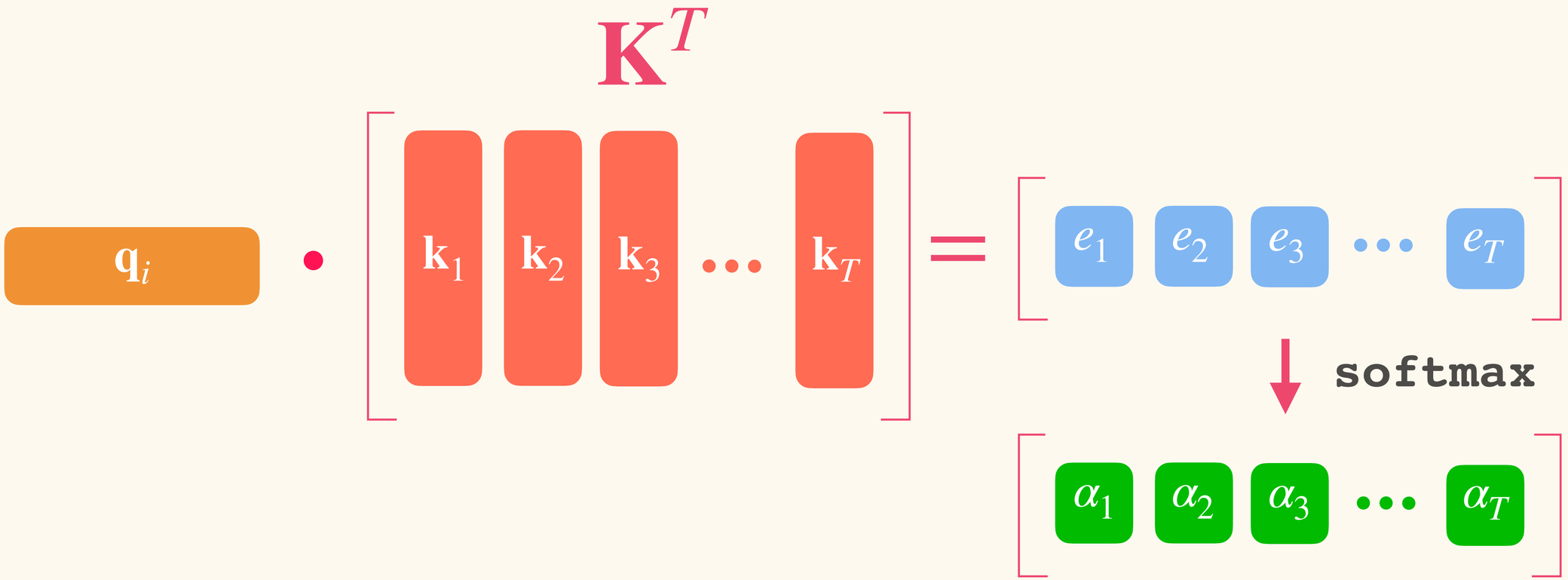
e_j 是 attention 的強度, Google 風是用內積:

$$e_j = \mathbf{q}_i \mathbf{k}_j^T$$

$e_1, e_2, \ldots, e_T$ 經 softmax 得 $\alpha_1, \alpha_2, \ldots, \alpha_T$, 最後令:

$$\mathbf{h}_i = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \cdots + \alpha_T \mathbf{v}_T$$

整個過程基本上是矩陣乘法



最後就是有名的 transformer 公式

注意基本上就是矩陣乘一乘就好了!

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_K}})V$$

d_K 是一個 key 向量的維度, 其實除以自己定的一個數就好, 這樣寫增加神秘感...

03.
生成模型為什麼這麼厲害呢?

Andrej Karpathy

Andrej Karpathy blog

About

The Unreasonable Effectiveness of Recurrent Neural Networks

May 21, 2015

There's something magical about Recurrent Neural Networks (RNNs). I still remember whe
recurrent network for Image Captioning. Within a few dozen minutes of training my first ba
arbitrarily-chosen hyperparameters) started to generate very nice looking descriptions of
the edge of making sense. Sometimes the ratio of how simple your model is to the quality
out of it blows past your expectations, and this was one of those times. What made this res
time was that the common wisdom was that RNNs were supposed to be difficult to train (with
I've in fact reached the opposite conclusion). Fast forward about a year: I'm training RNNs all the time and I've witnessed their power and robustness many times, and yet their magical outputs still find ways of amusing me. This post is about sharing some of that magic with you.

引發大家對文字生成 AI 興趣的名作。

http://karpathy.github.io/2015/05/21/rnn-effectiveness/

RNN: AI 數學家

To prove study we see that $\mathcal{F}|_U$ is a covering of $\mathcal{X}'$, and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X/}$ $i > 0$ and $\mathcal{F}_p$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$-modules on $\mathcal{C}$ as a $\mathcal{F}$-m In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\widetilde{M}^\bullet = \mathcal{I}^\bullet \otimes_{\mathrm{Spec}(k)} \mathcal{O}_{S,s} - i_X^{-1}\mathcal{F})$$

is a unique morphism of algebraic stacks. Note that

$$\mathrm{Arrows} = (Sch/S)^{opp}_{fppf}, (Sch/S)_{fppf}$$

and

$$V = \Gamma(S, \mathcal{O}) \longmapsto (U, \mathrm{Spec}(A))$$

is an open subset of X. Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S.

Proof. See discussion of sheaves of sets. □

電腦覺得自己是個數學家!

電腦產生代數幾何介紹 "Stacks" 的片段!

AI 生成的「莎士比亞」作品

達勒斯 (PANDARUS)：
唉，我想他應該快要被接近了，當那一點點的微雨將會在從未被餵養中被達成，而誰只是一個鏈條和他死亡的臣民，我不應該睡覺。

第二位元老 (Second Senator)：
他們離開了這份苦難，在我的靈魂上產生，破碎且強烈地應該被埋葬，當我滅亡大地和許多國家的思想。

文森西奧公爵 (DUKE VINCENTIO)：
好，你的機智是在側面和那個的照顧中。

第二位領主 (Second Lord)：
他們會在這個房間之後被統治，而且我美麗的訊息從事實中開始，被傳達，那些高貴的靈魂，我將擁有戰爭的心。

看完一本紅樓夢的 AI

紅樓夢生成器

起個頭, 幫你完成一段紅樓夢。可以改變 temperature, 越小生出的字越固定, 越大越隨機。

TEXTS

自孫悟空從石頭中蹦出來之後，

N CHARS

200

TEMPERATURE

1

Clear　Submit

OUTPUT

自孫悟空從石頭中蹦出來之後，說：「俗，先中用佛嘴罷。」雨村道：「寶玉既回來，又是聞世，也是知道奇異而不必丟。」士隱道：「老先生莫怪拙言，貴族之女，俱屬再聞？據伊禍，我們家兄可傳，舛錯，不如我再有細產，今日，老爺仍是接世。今日弟引近因。」說得賈政嘆了口氣。王夫人這樣一回，說：「老太太給他打聽，打聽發定主意，只要在家裡就完了。」邢夫人說他這樣一個，那裡都願意跟著，毫無蹤跡。裡頭邢夫人更哭著打聽「妞兒不用著了急，她榮兩府的人，也不

Flag

Prompt: 自孫悟空從石頭中蹦出來之後...

自孫悟空從石頭中蹦出來之後，先飛報單的。賈政嘆口氣。手又賞了一本，與他老先邀雨村回話。

那瓦礫明乍寒了一番補入，毫不進，說著：「悟如從此，從天入三洞天。因開燈，古道今日，原來如今的奇處，顏色大大道如此。」士隱自趕的道：「二老爺通報仙草已經失了那仙翁，你把這條猩紅氈的斗篷，向小子同賈珍珠方相認，俟薛姨媽那邊設停寺內。

次日，鳳姐等分頭取藥那劉姥姥家的，眾人打發人去給邢夫人辭了眾人。王夫人命賈蘭跟了侍兒去，分頭只一顧頭回來看著賈環的相思。王夫人便告訴了平兒，在賈芸家外沒有，不敢答言，只有好心盛妝。

那空空牢牢記著此言，又把五更五更收心起題，寫著書去。回奏：「皇上隆恩，內眷們不及大理的。」賈政帶著賈蘭場中迷失。

Karpathy 認為 AI 生成的都是「幻覺」

在他 2015 年著名的部落格文章 "The Unreasonable Effectiveness of Recurrent Neural Networks" 把 AI 生成的叫 **hallucinated (幻覺的)** 版本。

Karpathy 認為幻覺是 LLM 最偉大的特點

Andrej Karpathy
@karpathy

On the "hallucination problem"

I always struggle a bit with I'm asked about the "hallucination problem" in LLMs. Because, in some sense, hallucination is all LLMs do. They are dream machines.

We direct their dreams with prompts. The prompts start the dream, and based on the LLM's hazy recollection of its training documents, most of the time the result goes someplace useful.

It's only when the dreams go into deemed factually incorrect territory that we label it a "hallucination". It looks like a bug, but it's just the LLM doing what it always does.

At the other end of the extreme consider a search engine. It takes the prompt and just returns one of the most similar "training documents" it has in its database, verbatim. You could say that this search engine has a "creativity problem" - it will never respond with something new. An LLM is 100% dreaming and has the hallucination problem. A search engine is 0% dreaming and has the creativity problem.

- 從某種意義上來說, 幻覺正是大型語言模型 (LLMs) 所做的全部。它們是作夢的機器。
- 搜索引擎是 0% 的夢境, 但不會創造。
- 人們實際上的意思是, 不希望一個 LLM 助理發生幻覺。
- LLM 沒有「幻覺問題」。幻覺不是一個錯誤，它是 LLM 最偉大的特點。

文字生成 AI 的小總結

訓練上，大型語言模型就是要「很會接話」。

Karpathy 曾經說 AI 生成的叫「幻覺」版本。

白話文說，我們就是把它訓練成「唬爛王」。

幻覺有可能是對的，有可能是錯的，甚至無所謂對錯。

震驚世界的 GPT 唬爛王系列

SYSTEM PROMPT (HUMAN-WRITTEN)

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

MODEL COMPLETION (MACHINE-WRITTEN, 10 TRIES)

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

著名的「發現獨角獸」唬爛文章。

Better Language Models and Their Implications

https://openai.com/blog

GPT-2 參數量大到可怕!

GPT-2

15 億!

紅樓夢生成器

300萬

唬爛王第三代 GPT-3

大到無法下載! 只能申請 API 使用。

1,750億

唬爛功力更是前所未見!

正常人一生讀多少字呢?

ChatGPT 算錯了! 總之差不多 2 億字。

假設一個人每天花費30分鐘閱讀，每分鐘可以閱讀250個字，那麼一年下來，這個人可以讀書約27萬字。如果這個人一生閱讀時間持續這樣，那麼他或她在70年的壽命中可以讀書約5300萬字。

GPT-3 大約精讀了 4990 億的字,
一般人要活 9415 次才能達成!

- Gemini 說 10 億字
- Claude 說 3 億字
- Grok 說 2-2.5億,
  認真閱讀的 5 億

只是預測下一個字, 但看過超級多東西!

炎 龍 老 師 很 → → 帥

人類一輩子，包括各種閱讀，讀的字數大約是 1B 量級 (10 億)。

Llama 3 大約用了 15T 量級的訓練資料，是人類一輩子讀的 1.5 萬倍!

現代的 LLM 大概什麼都讀過了，所以什麼話都會接。

LLM 大型語言模型

大型語言模型

Large Language Model

五大商業模型

ChatGPT OpenAI

Gemini Google

Claude Anthropic

Grok xAI

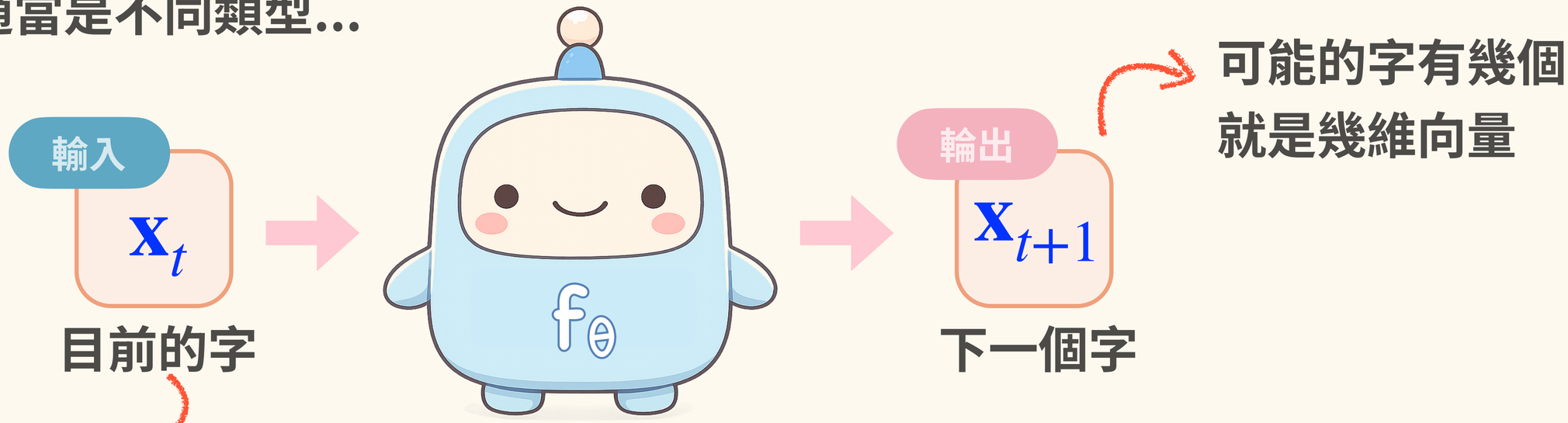
Muse Spark Meta

04.

選字的溫度

文字生成模型輸入和輸出的樣子其實不同

在 NLP「前一個字預測下一個字」的生成模型, 輸入和輸出通當是不同類型...

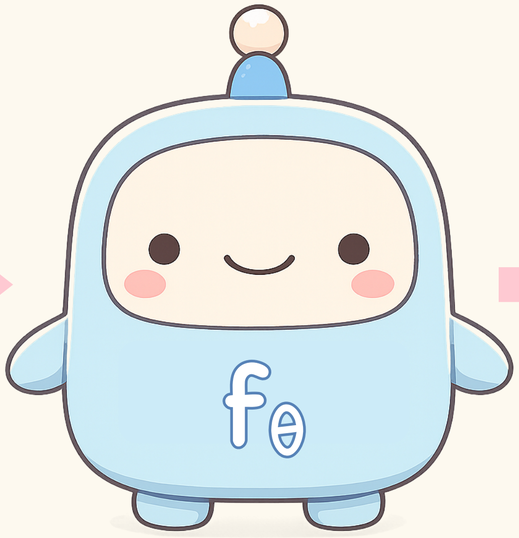


通常是某種代表向量
(word embedding)

接下來我們先討論輸出部份

模型其實是把每個可能的字評分

炎 龍 老 師 很



帥 1.40
87

幽 0.03
94

有 1.71
112

熱 −0.73
174

受 0.95
221

模型其實是把每個可能的字評分

		指數轉換	計算佔比
帥	1.40	4.05	30%
幽	0.03	1.03	7%
有	1.71	5.53	40%
熱	−0.73	0.48	4%
受	0.95	2.59	19%

87
94
112
174
221

softmax

選字原來也是有學問的!

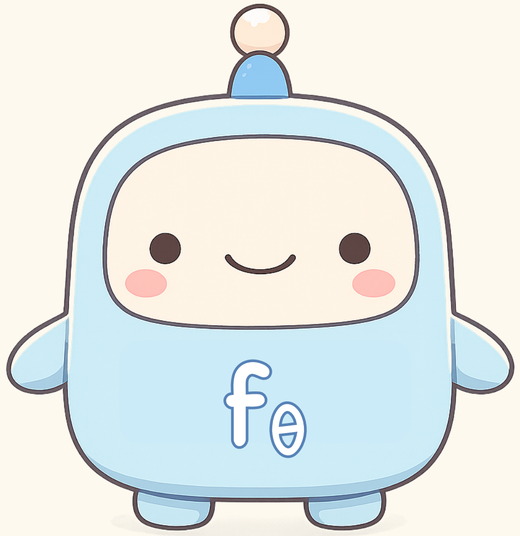
temperature

top-k

top-p

隨機選字的策略, 大概有調整 temperature, top-k, top-p 等等方式。

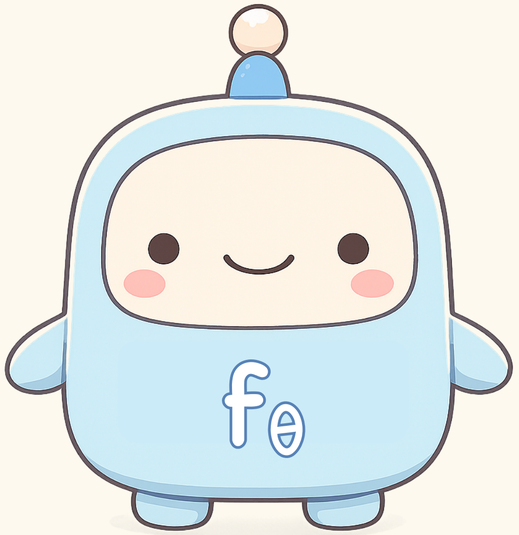
最重要的是依機率抽樣



a	0.7
b	0.29
c	0.01

這樣 70% 的機率選到 a 號字, 29% 會選 b 號字, 只有 1% 選 c 號字。

注意 softmax 是「贏者通吃」！



a	4.3	0.7
b	3.4	0.29
c	0.2	0.01

softmax

這裡會發現, 原來前兩名感覺沒差那麼多!

關鍵在做了指數轉換

$$a \rightarrow a' = e^a$$

$$b \rightarrow b' = e^b$$

$$c \rightarrow c' = e^c$$

a 經 softmax 轉換為:

$$\frac{a'}{a' + b' + c'}$$

所以常用 temperature 來調整

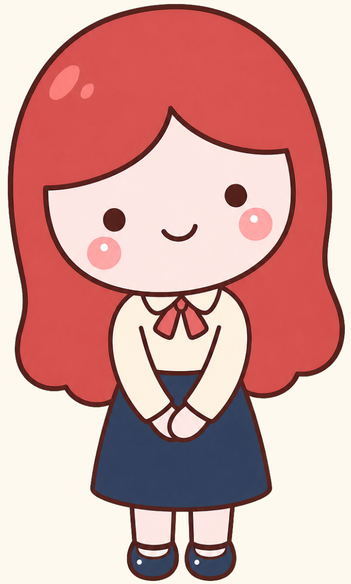
$$a \rightarrow a' = e^{a/\tau}$$

$$b \rightarrow b' = e^{b/\tau}$$

$$c \rightarrow c' = e^{c/\tau}$$

因此 $\tau > 1$ 時會拉近彼此的機率 (更隨機); 而 $\tau < 1$ 時機率更拉開 (也就是生成文字更固定)。

溫度的效果



		$\tau = 1$	$\tau = 2$	$\tau = 0.5$
a	4.3	0.7	0.57	0.86
b	3.4	0.29	0.36	0.14
c	0.2	0.01	0.07	0.

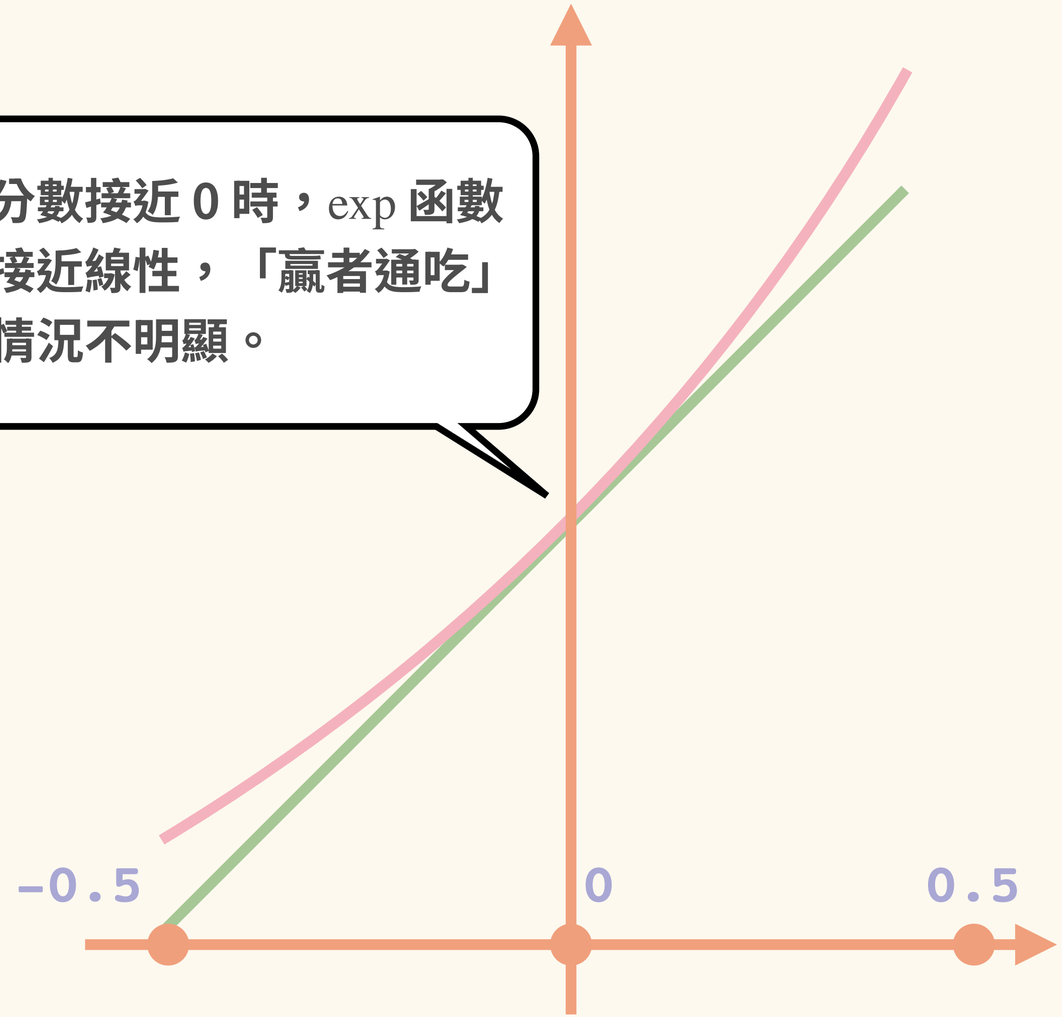
logit

$\tau > 1$ 會拉近分數、更隨機。

$\tau < 1$ 進一步拉開分數，更

事實上大型語言模型通常不用擔心

分數接近 0 時，exp 函數接近線性，「贏者通吃」情況不明顯。



LLM 輸出可能很多

最終大家分數都不高，也就是都很接近 0。

常見設定

實務上設 $\tau = 1$ 是常見的，就算要隨機一些也只設到 $\tau = 1.1$。反而有時希望每次輸出比較一致時會設 τ 是較小的數字。

不管如何，機率低的還是有可能被選到!!

這樣選字機率低的字雖然比較難被選到, 但還是有可能...

於是時常不知在亂說什麼...

top-k 只選前 k 名

我們先決定，比如 $k = 50$，這樣只有前 50 名的字會被選到。

top-k 還是會有問題

有時輸出的可能非常少，比方說「政大應數系教我們生成式 AI 的老師是蔡炎」，下一個字必須是「龍」。

Top-p 解決這個問題

先決定一個機率 (比如 $p = 0.9$)。

從機率最高的字開始加, 看什麼時候大於 p。

$$\sum_{i=1}^{N} p_i > p$$

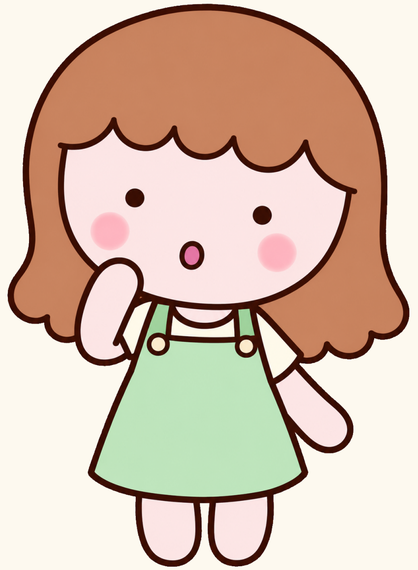
只選前 N 個字。

Redistribution

這時前 N 名的字機率加起來通常不是 1。

於是我們做「好高級」的 redistribution...

$$p'_k = \frac{p_k}{P}$$



* 這裡的 $P = \sum_{i=1}^{N} p_i$

最讓人驚訝的是 top-p 很晚才出現

不要沒禮貌, top-p 正式名稱叫 nucleus sampling, 是 AI 頂會 ICLR 2020 的文章。

* Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, Yejin Choi, “The Curious Case of Neural Text Degeneration,” ICLR 2020

總之文字生成是一堆字生一個字的模型

總之, 生成模型就是用前面的字, 生出一個機率分佈, 再選一個策略, 依此 sample 出下一個字。

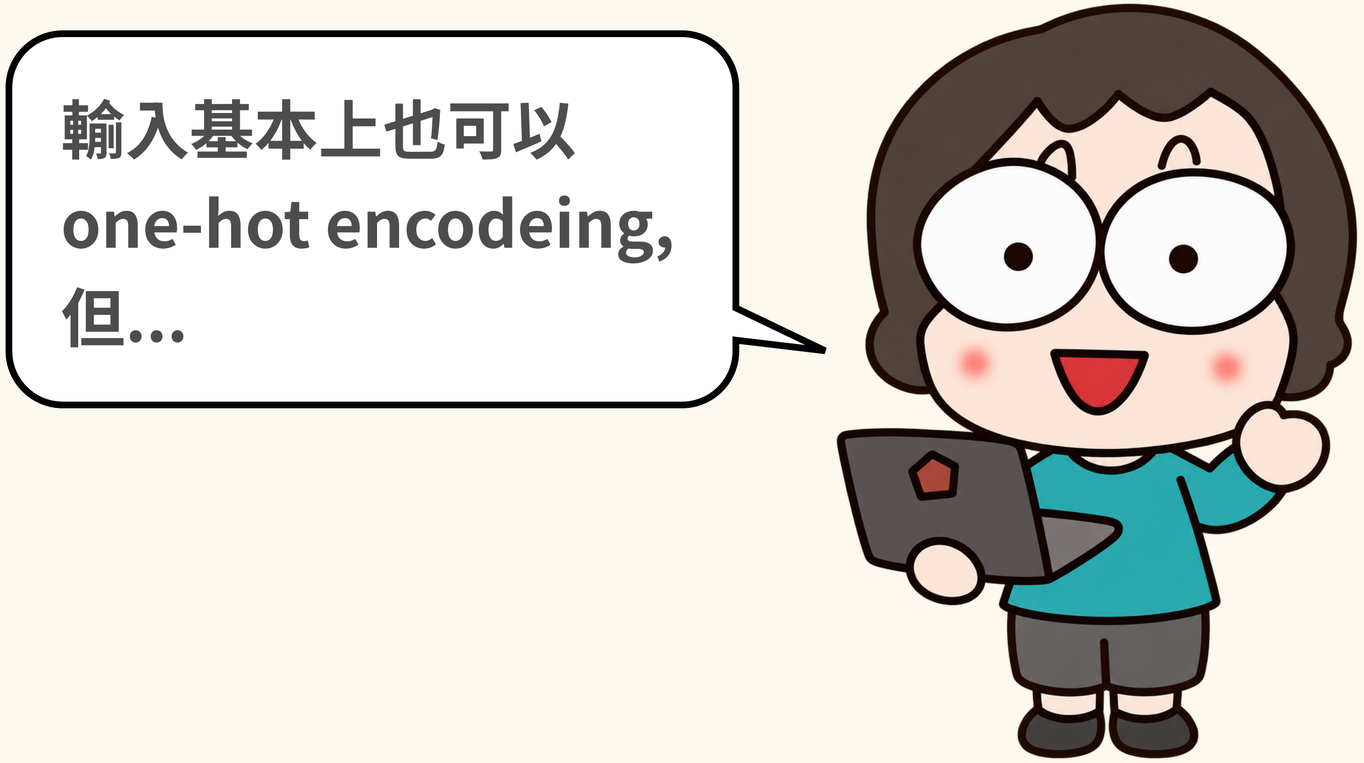
$$P_\theta(\mathbf{w}_i \mid \mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T)$$

其中 $\mathbf{w}_i$ 是我們字庫裡的第 i 個字。

05.

尋找特徵的小任務

輸入比較出複雜一點



當然也可以用 One-Hot Encoding, 但...

前面說每個字有個編號, 而代表向量就用 one-hot encoding。

1 的 $\begin{bmatrix}1\\0\\0\\0\\0\\\vdots\end{bmatrix}$

2 一 $\begin{bmatrix}0\\1\\0\\0\\0\\\vdots\end{bmatrix}$

3 了 $\begin{bmatrix}0\\0\\1\\0\\0\\\vdots\end{bmatrix}$

4 是 $\begin{bmatrix}0\\0\\0\\1\\0\\\vdots\end{bmatrix}$

5 我 $\begin{bmatrix}0\\0\\0\\0\\1\\\vdots\end{bmatrix}$

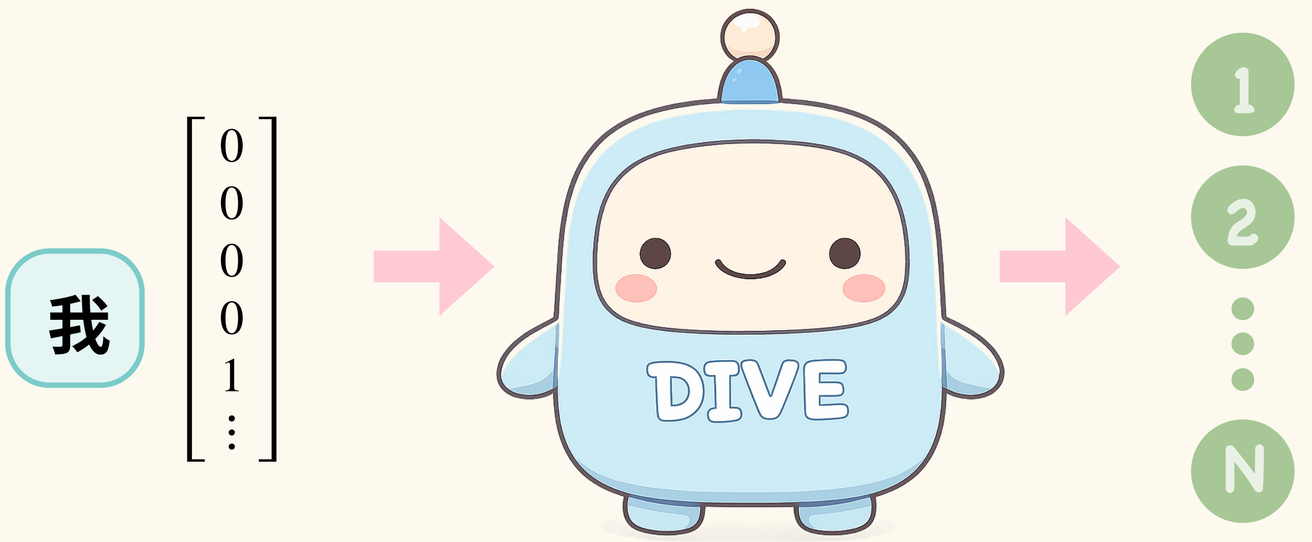
注意 one-hot encoding 後還是只是個編號!

最主要希望有點「意思」

可不可以輸入帶點文字的「意思」？

尋找特徵代表向量

一個字 (或一個詞), 我們想給一個比較有意義的「特徵代表向量」。

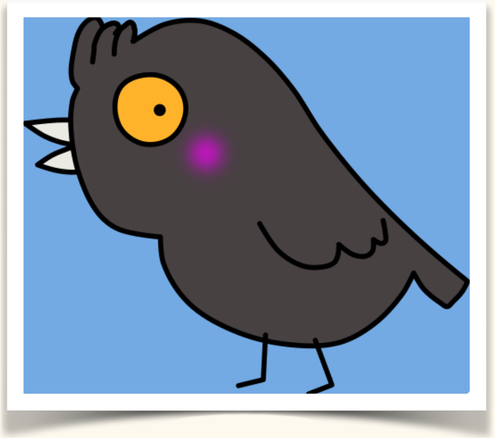


這樣的模型就叫做一個 word embedding。

提升 AI 素養到下一個層次的秘密

這是 Level 2 的 AI！

假設我們有個訓練得很好的八哥辨識 AI

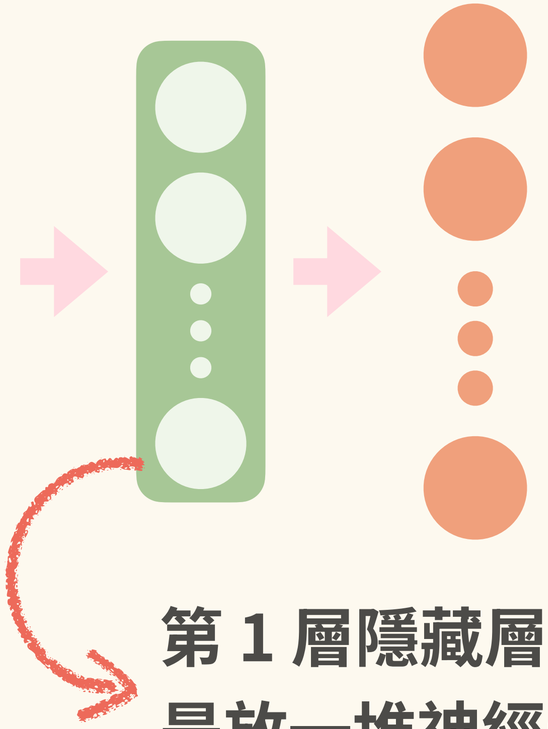


輸入

1	0.70
2	0.15
3	0.05

輸出

每一層神經網路把輸入轉為另一個 tensor

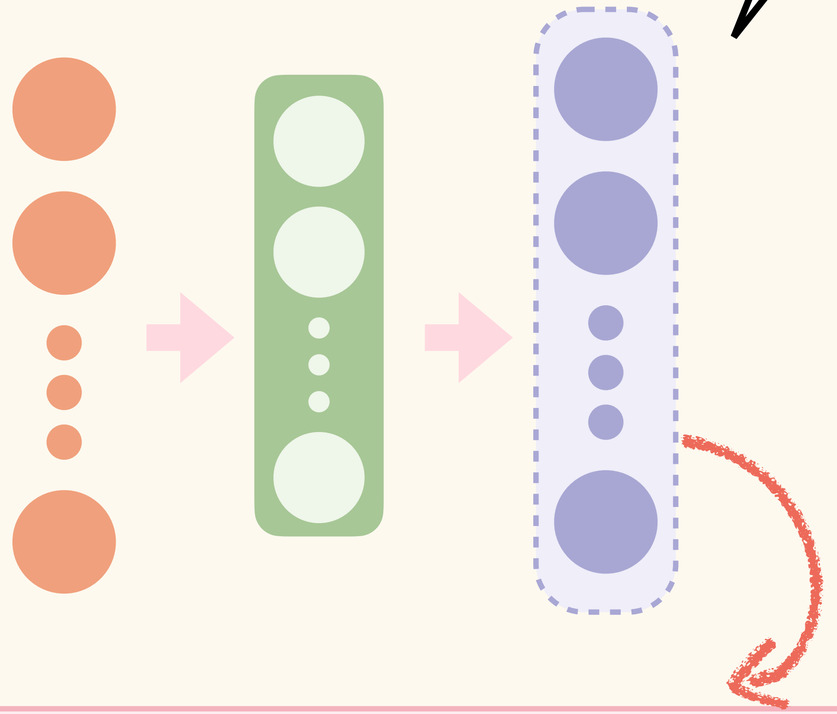


輸出通常是一個向量。

第 1 層隱藏層, 基本上就是放一堆神經元在裡面

每一層神經網路把輸入轉為另一個 tensor

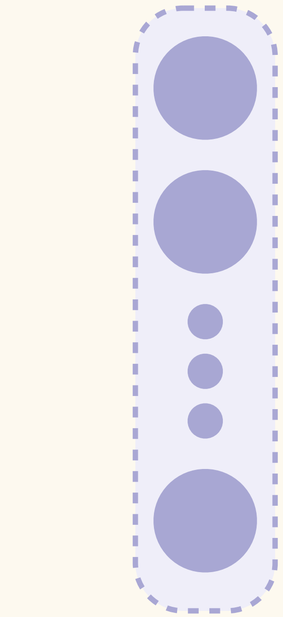
上一層的輸出就是下一層的輸入，如此不斷下去...



這是代表輸入那隻土八哥的「特徵代表向量」。

每一層的輸出，都可以當成電腦對輸入的**理解**

Embedding



特徵代表向量

1 0.70

2 0.15

3 0.05

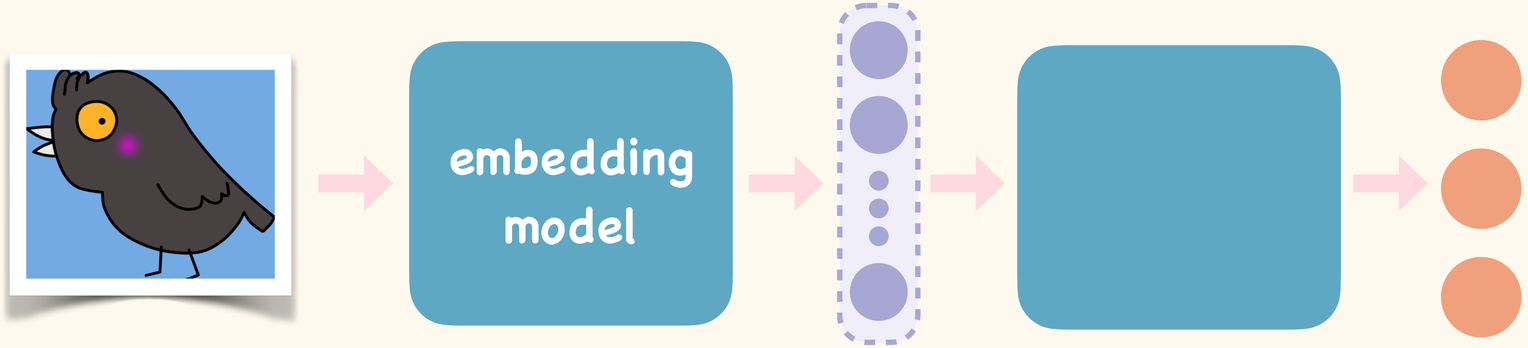
輸出

最後這些數字真的正確辨識是口哪種 AI，所以我們可以當成很好輸入的特徵代表向量。

這個特徵代表向量，常被我們稱為輸入的 embedding。

代理任務 (Pretext Task)

我們可以讓電腦去做一些小任務, 這個任務是我們覺得「電腦要懂輸入」才能完成的任務。這種不是我們真正最後的目標, 通常是為了訓練好的表示向量的叫 pretext task。



我們看要嵌入到幾維向量, 比如說 N=128 維, 那就在神經網路中間的隱藏層, 放 128 個神經元!

Word2Vec 用了兩種不同的小任務

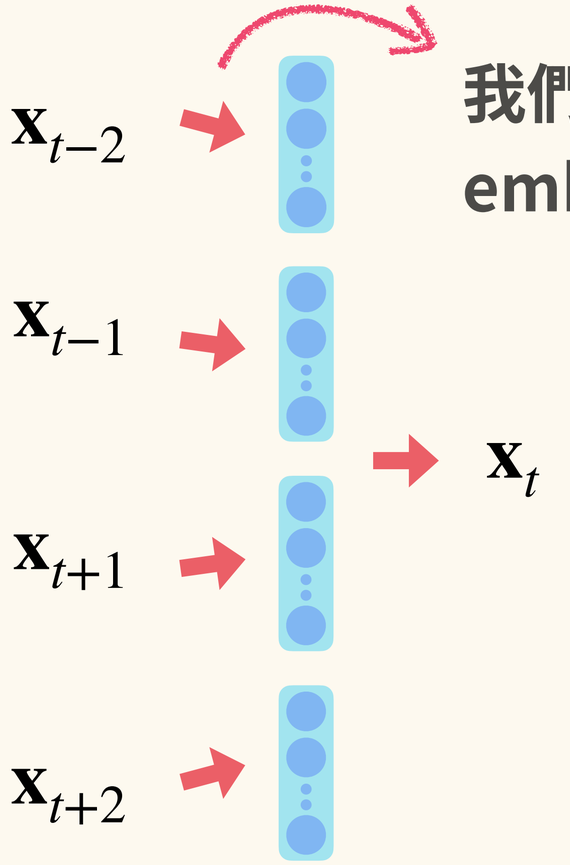
Word2Vec 就設計兩種任務。

CBOW model

用周圍的字預測中間的字。

$\mathbf{x}_{t-2}$	$\mathbf{x}_{t-1}$	?	$\mathbf{x}_{t+1}$	$\mathbf{x}_{t+2}$

訓練成功, 某層隱藏層輸出當 embedding



我們可以找到字的 embedding!

CBOW model

中間的字預測旁邊的會不會更厲害?

或是更炫的去訓練這樣的函數!

Skip-Gram

中間的字預測週圍的字

訓練完電腦好像「真的有懂」

結果會發現電腦好像「真的有懂」。

Google 官網: https://code.google.com/archive/p/word2vec/

標準的火力展示

訓練好了有很多炫炫的功能。

巴黎 ＋ 法國 － 義大利 ＝ 羅馬

國王 － 男人 ＋ 女人 ＝ 皇后

T. Mikolov, K. Chen, G. Corrado, J. Dean. Toutanova. Efficient Estimation of Word Representations in Vector Space. Proceedings of Workshop at ICLR, 2013..

在 Word2Vec 中權重剛好是「記憶」

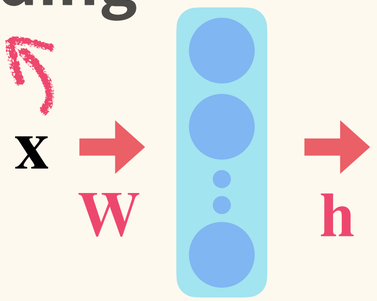
$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1V} \\ w_{21} & w_{22} & \cdots & w_{2V} \\ \vdots & \vdots & & \vdots \\ w_{i1} & w_{i2} & \cdots & w_{iV} \\ \vdots & \vdots & & \vdots \\ w_{N1} & w_{V2} & \cdots & w_{NV} \end{bmatrix}$$

word2vec 給我們一個啟發, 就是權重也可以是我們日後要用的部份, 可以當成是某種「記憶」。

其至這一小段網路想成是「理解」。

原因其實很簡單

One-hot encoding



$$W^T \cdot x = h$$

就 word2vec 來說, 其實也可以看成是隱藏層的輸出!

$$\begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1N} \\ w_{21} & w_{22} & \cdots & w_{2N} \\ \vdots & \vdots & & \vdots \\ w_{i1} & w_{i2} & \cdots & w_{iN} \\ \vdots & \vdots & & \vdots \\ w_{V1} & w_{V2} & \cdots & w_{VN} \end{bmatrix}^T \cdot \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} = h$$

注意 transformer 其實有自己的 embedding 層

Trasnformers 用 one-hot encoding 輸入後, 會連到自己的 embedding 層, 也就是訓練時會同時訓練 word embedding。

05.

使用大型語言模型

五大商業模型

ChatGPT OpenAI

Gemini Google

Claude Anthropic

Grok xAI

Muse Spark Meta

可以考慮的開源型 LLM

超大型

- GLM-5.3 753B
- Kimi 3 2800B

注意一般人基本上跑不動!

中小型

- Qwen3.8 27B
- Muse Glimmer 30B
- Gemma4 31B/26B

超輕量

- K2 Horizon 7B
- Gemma4 12B
- Qwen3.5 4B/9B

怎麼找適當的模型呢？

Arena AI

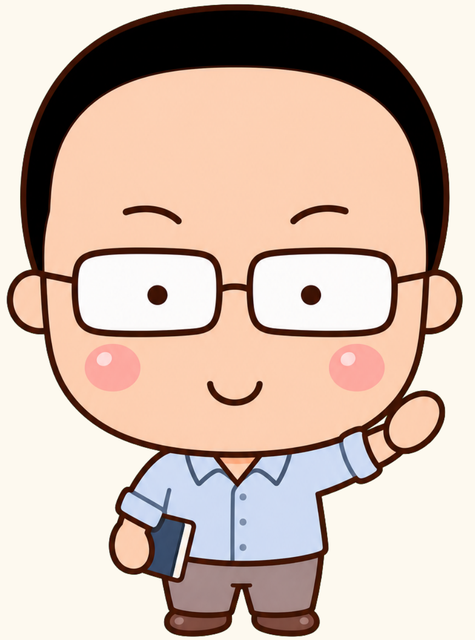
https://arena.ai/

Artificial Intelligence

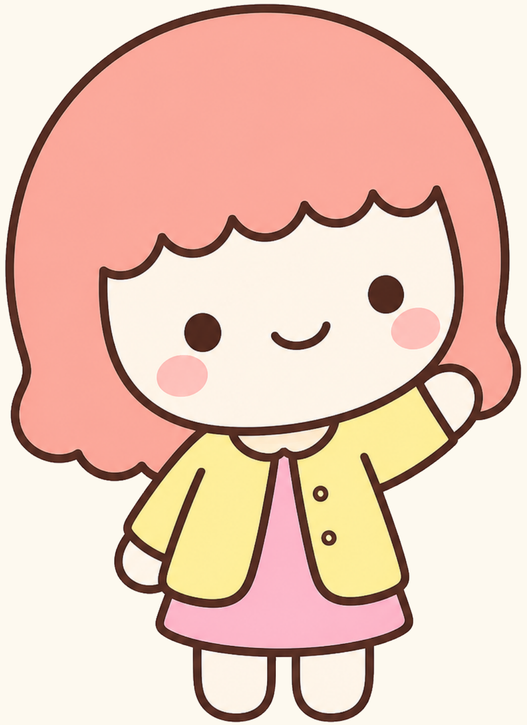
https://artificialanalysis.ai/

Benchmark 分數高的模型一定比較好?

LLM 一般的 benchmarks 當然有其意義, 但相信我們也不一定那麼在意, 自己慣用的 LLM 是不是比另外一個會解數學題。



設計幾個標準測試 prompts



這些問題最好你**相當懂**
— 至少不能完全不懂，
你才可以知道他回答得
如何。

怎麼用開源的 LLM 呢?

LM Studio

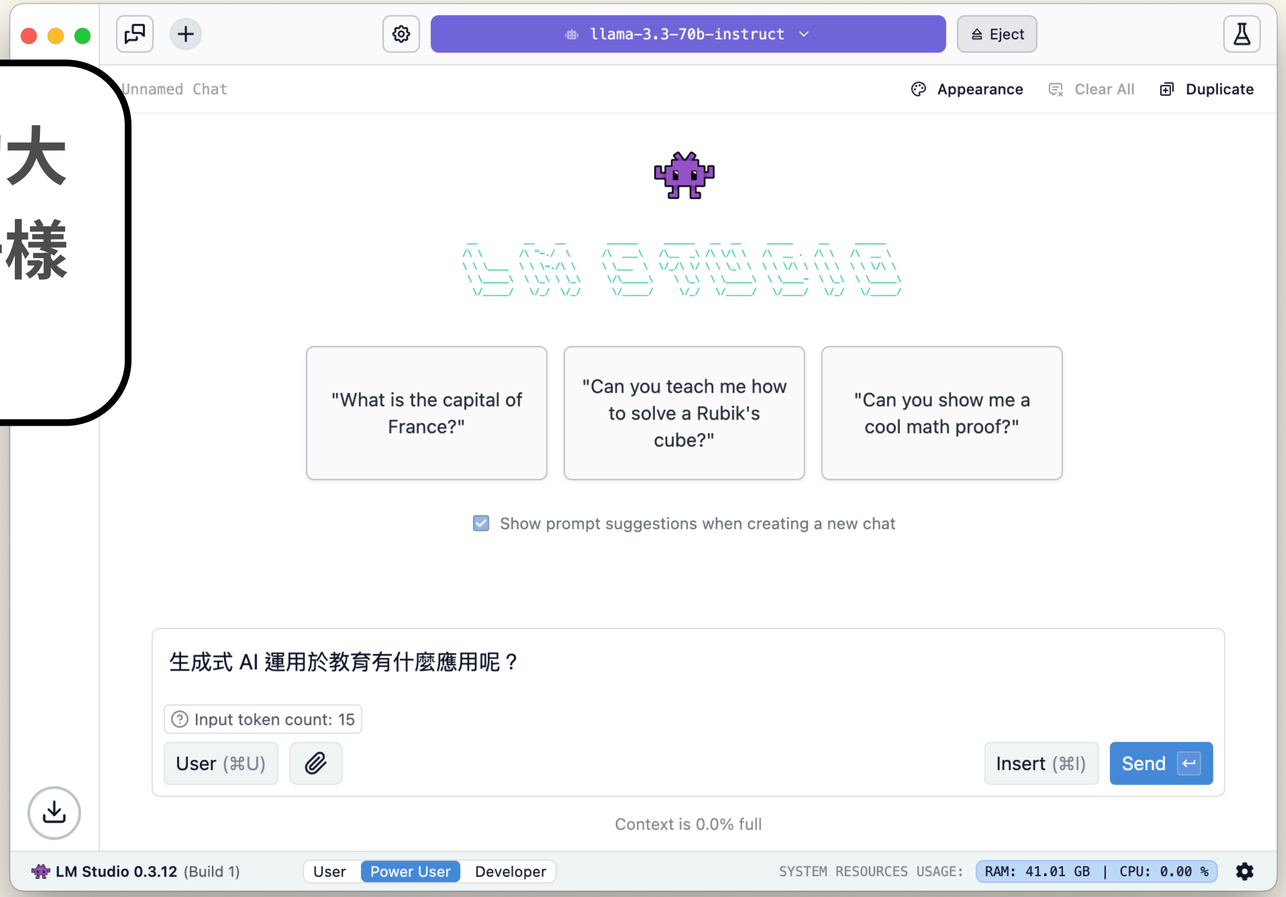
一般人的選擇

Ollama

簡潔、終端機介面

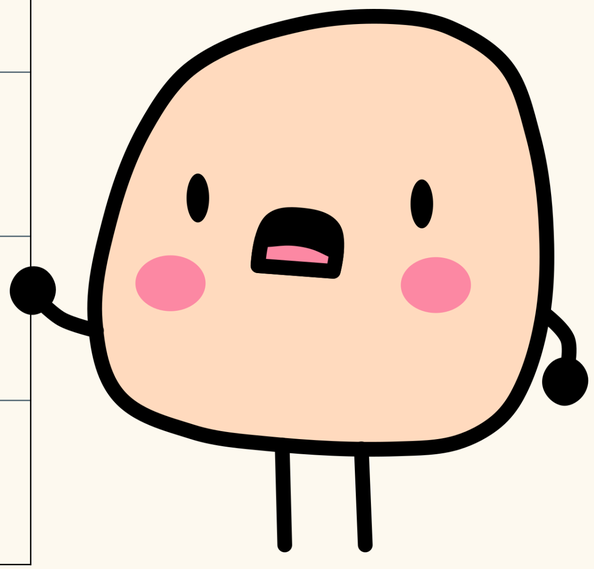
LM Studio

和很多閉源的大型語言模型一樣的界面。



跑模型的機器考量 (NVIDIA GPU)

GPU	VRAM
RTX 5090	32GB
RTX 4090	24GB
H100	80GB
H800	80GB



模型大小

gpt-oss 20B

200 億個參數

要多大的 VRAM 才可以跑呢?

以 4-bit 量化的版本為例。

$$0.5 \times 20 = 10G$$

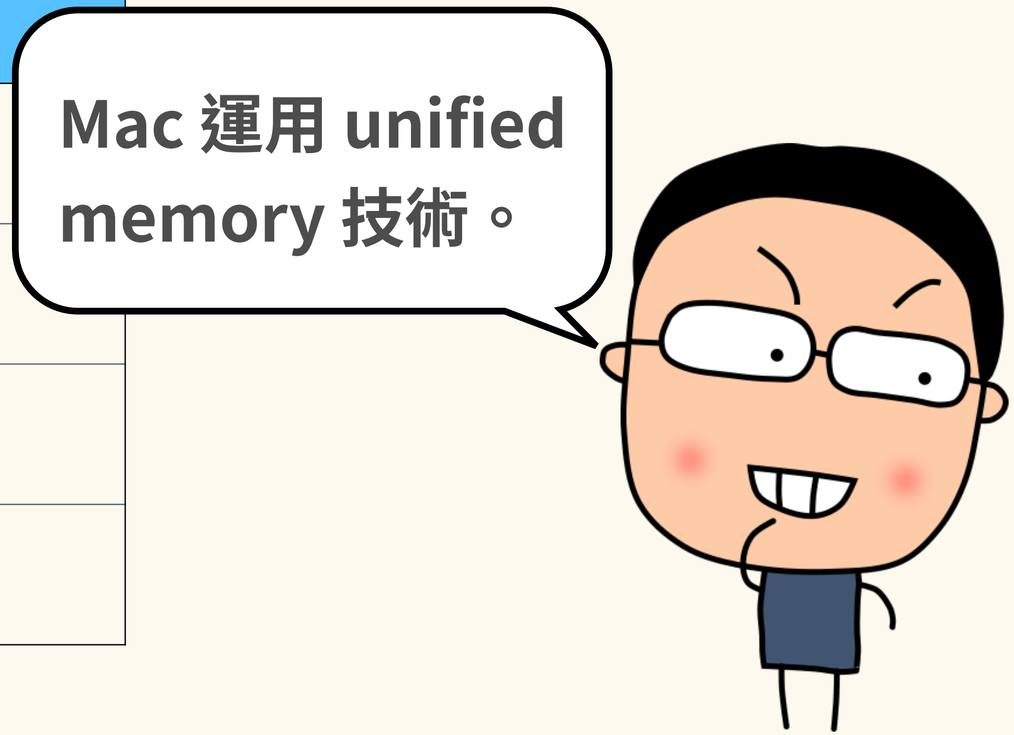
模型大小

要超過

實際建議 16G 可跑

Mac 有多大的記憶體，基本上有多大的 VRAM!

機型	最高 RAM
Mac Mini	64GB
MacBook Pro	128GB
Mac Studio	512GB
Mac Pro	192GB



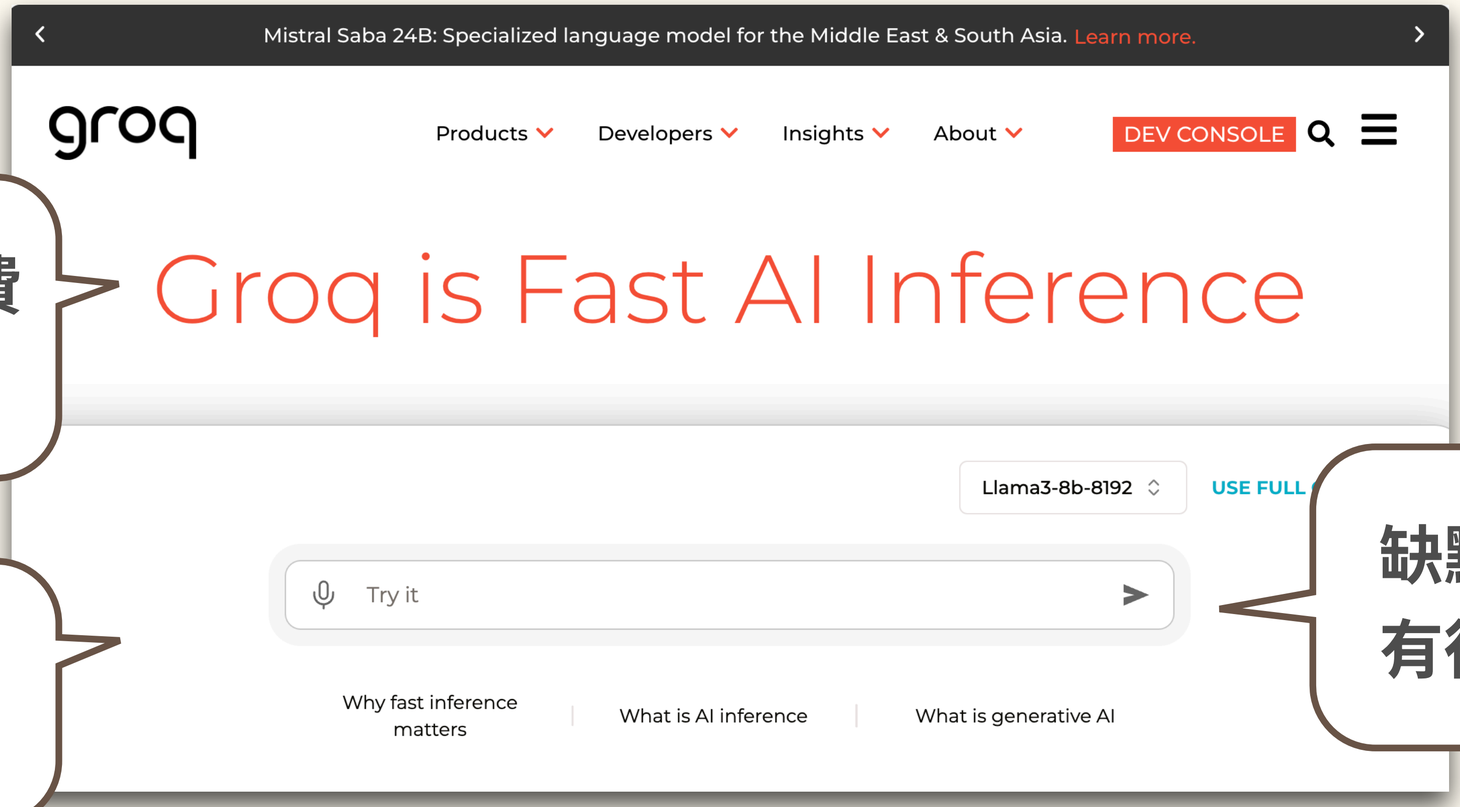
共用記憶體非 Mac 篇

AMD Ryzen AI Max+ 395

NVIDIA DGX Spark

這些都是 128GB
共用記憶體。

用開源也不一定要裝在自己電腦



少見可以完全免費使用 API。

速度很快!

缺點是模型並沒有很多。

https://groq.com/

另外推薦 Ollama Cloud

Ollama

在雲端跑 Ollama (需訂閱)。

跑超大開源模型

像 GLM-5.3，Kimi 3 等等超大模型不用在自己機器上跑。

用法就平常用法

```
ollama pull kimi-k3:cloud
```

06.
Prompt 必殺技！

寫好 Prompt 只有兩個重點

正確的資訊

它不知道你腦中的背景——把該給的資料、條件講清楚。

清楚的指引

要它做什麼、輸出成什麼樣子、給誰看，講明白。

兩個重點而已：
正確的資訊 ＋ 清楚的指引。

這次作業，其實就是一次 Context Engineering

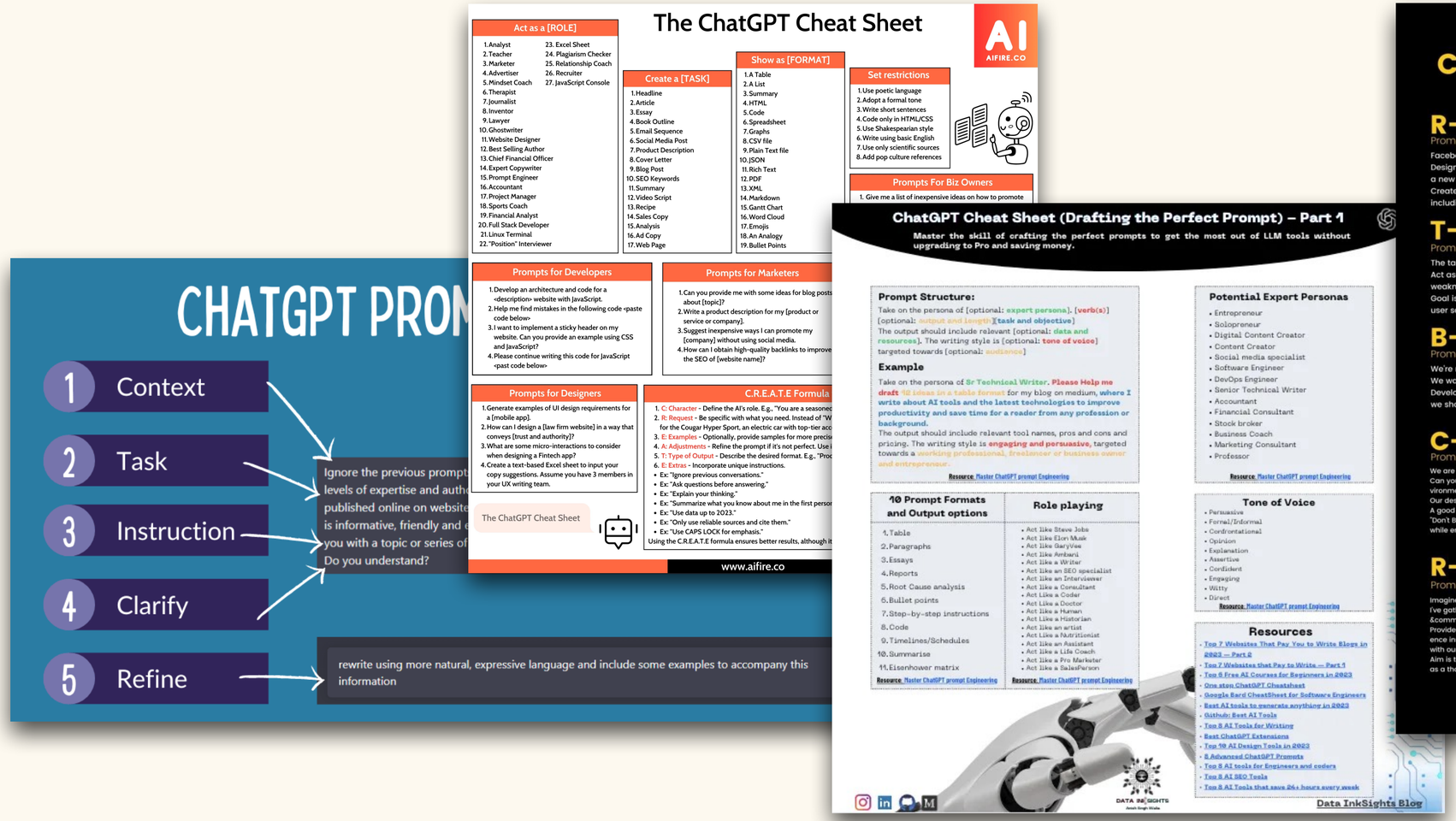
Chat Bot

- Prompt Engineering
- Context Engineering
- Harness Engineering
- Loop Engineering
- Graph Engineering

AI Agent

同時練習了 prompt engineering 和 context engineering

瞭解原理，會成為一個強大的吸收器!



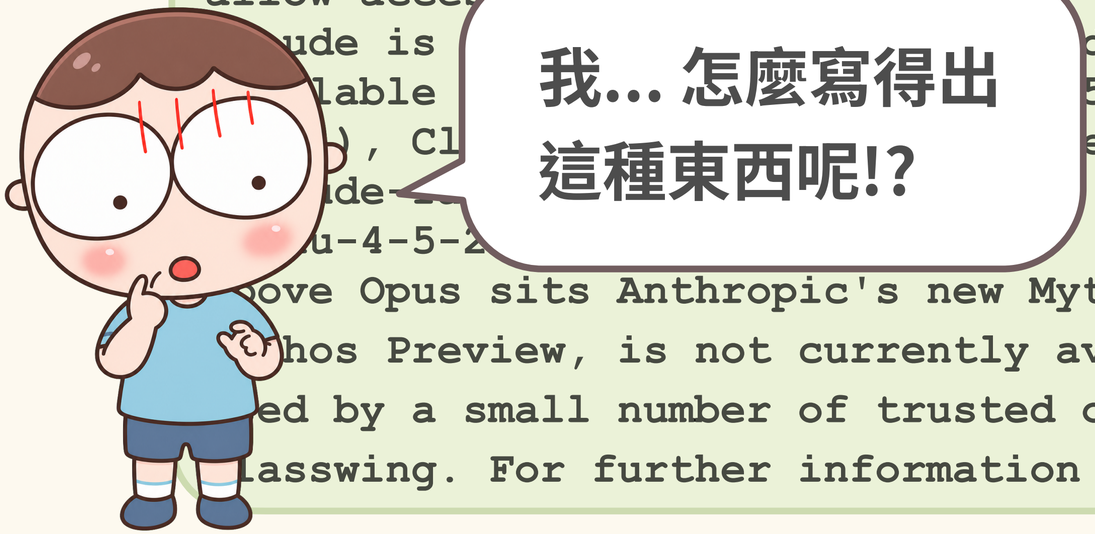
自己越強, AI 就越強!

- AI 可以提供客服人員 20% 的生產力
- 對需要產生文案、翻譯、報告、整理簡報及會議記錄的人, 增加 70% 的生產力
- 程式設計師更提升 125% 的生產力
- 甚至芥川賞作者九段理江都說她小說 5% 是 ChatGPT 寫的

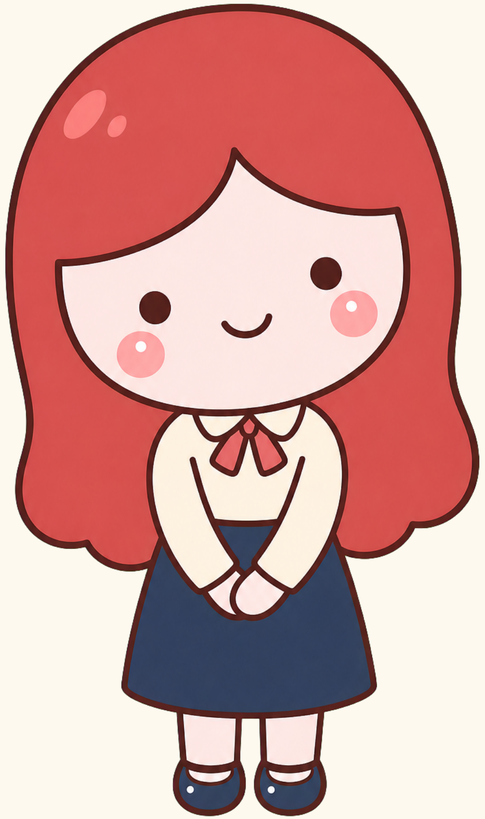
現代人寫 prompt，真的像作文比賽

Claude Opus 5 System Prompt

<claude_behavior> <product_information> Here is some information about Claude and Anthropic's products in case the person asks:
The currently selected version of Claude is Claude Opus 5. Claude Opus 5 is a powerful model for complex challenges.
Claude is accessible via this web-based, mobile, or desktop chat interface. If the person asks, Claude can tell them about the following products which also allow acces
ude is d Claude Platform. The most recent publicly
lable 5, Claude Opus 5 (the currently selected
), Cl e Haiku 4.5. They use the API model strings
ude- 'claude-sonnet-5', and 'claude-
u-4-5-2
ove Opus sits Anthropic's new Mythos tier. The first Mythos-class model, Claude
hos Preview, is not currently available to the public. It is currently being
ed by a small number of trusted organizations as part of Anthropic's Project
lasswing. For further information on this topic, Claude can direct the person to



你不用擔心，因為 prompt 必殺技今天教你！



和 AI 一起寫 prompt，就是寫 prompt 的必殺技!

如果我們下了這個 prompt...

請介紹 K-POP 女團 IVE。

會得到一篇中規中矩，沒什麼感情的文章。

1

先試試一個陽春版。

把你先要說的告訴 AI

請用 DIVE (IVE 粉) 的角度，來介紹 K-POP 女團 IVE。要包括她們出道即巔峰、被封為「完顏團」。但也經過一些辛苦的時刻，像是首張正規專輯《I AM》宣傳期 Rei 生病沒有參與，只有五位成員打歌宣傳。還有在 2024 年雖然登上 Lollapalooza，完成第一次巡演，但感覺整個氣勢不如從前，甚至有「芙學家」黑粉的攻擊。因此 2024 是有點低迷的一年。而 2025 年《REBEL HEART》的歌詞，呈現了 IVE 的態度，也振奮了士氣有點低落的 DIVE 們。

2

補充只有你知道的內容

請 AI 訪問你

3

請 AI 訪問你

…

這樣的主題，你覺得還有什麼可以釐清的地方可以問我。

再請 AI 幫忙補充、整理成草稿

請依我們以上的討論, 再收集一下 IVE 有哪些是 K-POP 女團第一的紀錄, 然後請打好草稿 (也就是先不要寫文章), 把要寫的重點整理好, 方向整理好。

以後直接用這個 prompt，指引 AI 寫文章。

4

請 AI 幫忙查證、最後整理成一個 prompt

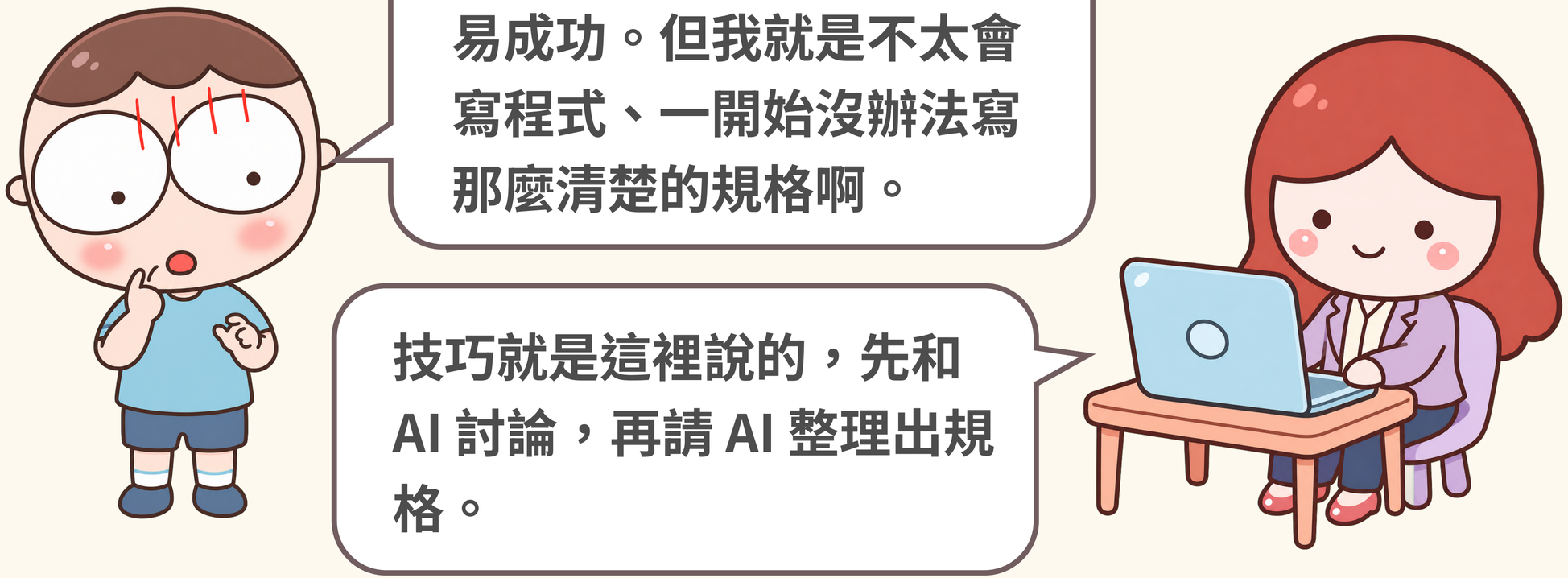
重點：AI 生成的、請務必要自己讀一次

5

親自閱讀並修改 prompt

只要是 AI 生成的，不管是 prompt 還是寫出的成品，一定要好好讀一讀!

這種小技巧在 Vibe Coding 也很好用!



【作業】

和 AI 一起設計一個真正好用的 Prompt

練習 prompt engineering 的作業

作業

和 AI 討論，共同設計出一份可以直接交給不同 LLM 使用的完整 prompt。

繳交指引

- ✅ 要交出你最後的 prompt，要特別說明哪些是你告訴 AI、怎麼和 AI 討論的。
- ✅ 最後的 prompt 「自己必需完全讀過並修改」。(可以說自己修改了什麼)
- ✅ 至少在三個 LLM 試用這個 prompt，紀錄自己的心得。

Q&A